



**The
University of
Faisalabad**

Lab Manual

Subject: Data Mining

Course Code DS-303

By

Muhammad Hassan 2023-BS-AI-176

Supervised by Sir Arham

Semester: 6th

The University of Faisalabad
Bachelors of Science in Artificial Intelligence
DEPARTMENT OF COMPUTER SCIENCE

Table of Contents

By.....	0
Lab 1.....	5
Knime Introduction	5
Practical Objectives:	5
Procedure:	6
Step 1. Create a New Workflow.....	6
KPI Summary:	9
Lab 2.....	10
CRISP-DM Methodology	10
Real World Problem	10
Dataset.....	10
Practical Objectives:	10
Procedure:	10
KPI Summary:	13
Lab 3.....	14
Data Cleaning.....	14
Real World Problem	14
Dataset.....	14
Practical Objectives:	14
Procedure:	14
KPI Summary:	17
Lab 4.....	18
Data Preprocessing	18
Real World Problem	18
Dataset.....	18
Practical Objectives:	18
Procedure:	18
KPI Summary:	21
Lab 5.....	22
Market Basket Analysis.....	22
Real World Problem	22
Dataset.....	22

Practical Objectives:	22
Procedure:	22
KPI Summary:	25
Lab 6.....	26
Apriori Algorithm.....	26
Real World Problem	26
Dataset	26
Practical Objectives:	26
Procedure:	26
KPI Summary:	29
Lab 7.....	30
FP-Growth Algorithm	30
Real World Problem	30
Dataset	30
Practical Objectives:	30
Procedure:	30
KPI Summary:	34
Lab 8.....	35
Python Basics for Data Processing	35
Real World Problem	35
Dataset	35
Practical Objectives:	35
Procedure:	35
KPI Summary:	41
Lab 9: Decision Tree Classification.....	42
Real World Problem	42
Dataset	42
Practical Objectives	42
Procedure	42
Step 1. Generate / load the dataset (Python)	42
Step 2. Encode categoricals and split the data	43
Step 3. Train and evaluate the Decision Tree	43
Step 5. Confusion matrix and feature importance	44

Lab 10: Naïve Bayes & KNN for Spam Detection	46
Real World Problem	46
Dataset.....	46
Objectives.....	46
Procedure.....	46
Step 1. Build the corpus and vectorise the text.....	46
Step 2. Train Naïve Bayes and KNN and measure FPR.....	47
Step 3. Compare confusion matrices and metrics visually.....	47
KPI Summary.....	48
Lab 11: Support Vector Machines for Fraud Detection.....	49
Real World Problem	49
Dataset.....	49
Objectives.....	49
Procedure.....	49
Step 1. Scale features and split (stratified).....	49
Step 2. Train Linear and RBF SVMs.....	50
Step 3. ROC curves and confusion matrices.....	51
Step 4. Visualise the non-linear boundary	52
KPI Summary.....	52
Lab 12: Clustering for Customer Segmentation	53
Real World Problem	53
Dataset.....	53
Objectives.....	53
Procedure.....	53
Step 1. Scale features and find the best k.....	53
Step 2. Fit K-Means and profile the segments.....	54
Step 3. Visualise segments and silhouette quality	55
Step 4. Hierarchical clustering and dendrogram	55
KPI Summary.....	56
Lab 13: Outlier Detection in Banking Transactions.....	57
Real World Problem	57
Dataset.....	57
Objectives.....	57

Procedure.....	57
Step 1. Scale data and define the evaluation.....	57
Step 2. Local Outlier Factor and Isolation Forest.....	58
Step 3. Anomaly-score distribution and comparison.....	59
KPI Summary.....	60
Lab 14: Web Scraping & Sentiment Analysis	61
Real World Problem	61
Dataset.....	61
Objectives.....	61
Procedure.....	61
Step 1. Scraping pattern and text cleaning	61
Step 2. VADER lexicon sentiment	62
Step 3. Supervised TF-IDF + Logistic Regression.....	62
Step 4. Key-word word clouds	63
KPI Summary.....	64
Lab 15: Final Capstone – End-to-End Patient Hospital Readmission Prediction (30 Days)	65
Dataset.....	65
Practical Objectives	65
Procedure.....	65
Step 1. Load and Preprocess Data.....	65
Step 2. Train and Compare Eight Models	66
Step 3. ROC Curves for All Models	66
KPI Summary.....	68

Lab 1

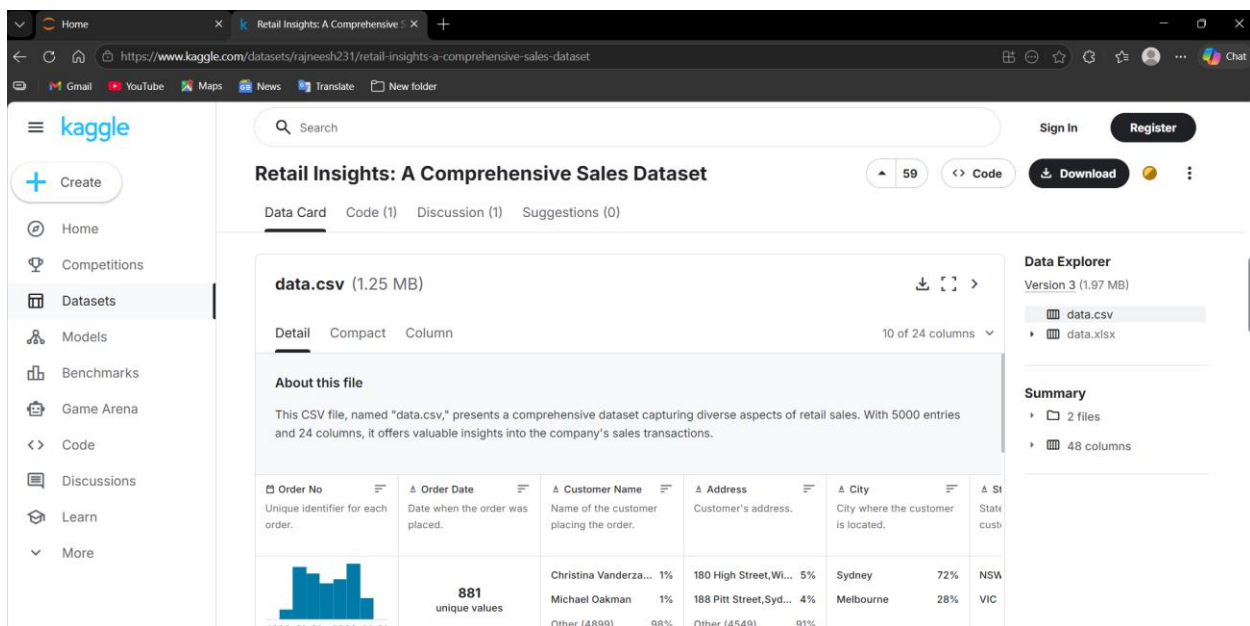
Knime Introduction

Real World Problem:

A retail store has accumulated sales data across multiple product categories and needs a structured way to analyse its performance. The management requires a clear understanding of overall revenue, the average value of each order placed, which categories drive the most income, and which individual products are the highest earners. This lab uses KNIME to address these needs through a series of visual, node-based operations on a CSV dataset

Dataset:

Retail Sales csv format dataset



Practical Objectives:

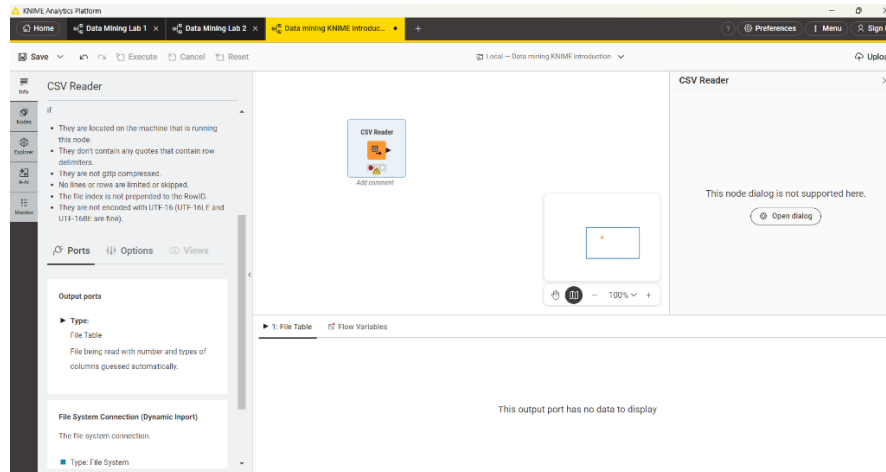
By the end of this lab, students will have hands-on experience with the core features of KNIME Analytics Platform. Specifically, students will be able to:

- Import a CSV dataset into a KNIME workflow using the CSV Reader node.
- Inspect and correct column data types where necessary.
- Generate descriptive summary statistics to understand data distribution.
- Create a derived Revenue column using the Math Formula node.
- Compute four business KPIs using GroupBy and Sorter nodes.

Procedure:

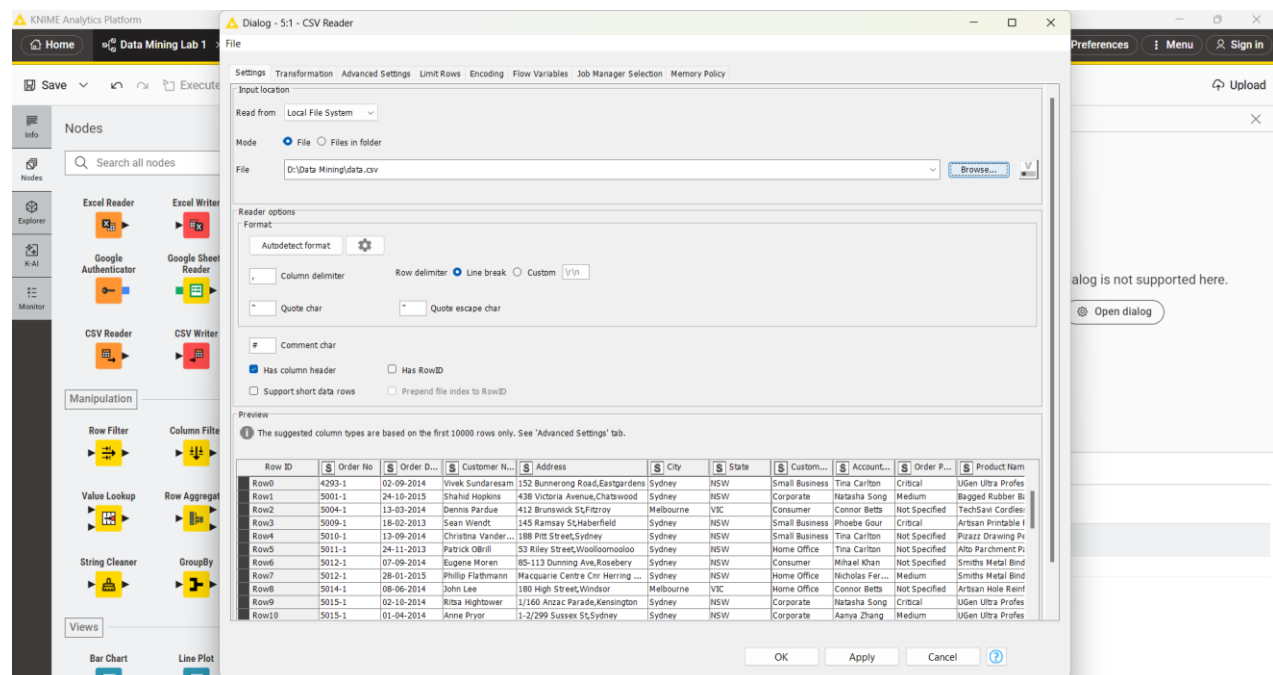
Step 1. Create a New Workflow

Launch KNIME Analytics Platform. From the menu, go to File ,New , New KNIME Workflow. Name the workflow Retail Sales Analysis and click Finish. This creates a blank canvas where all nodes will be placed and connected.



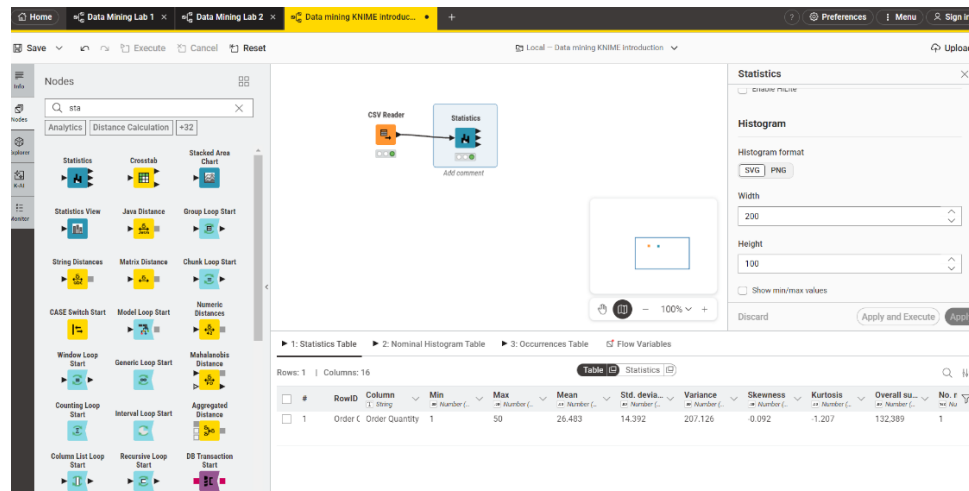
Step 2. Import the Dataset

Search for the **CSV Reader** node in the Node Repository panel on the left. Drag it onto the workflow canvas, then double-click it to open its settings. Click Browse and select the Retail Sales CSV file, then click OK. Right-click the node and select Execute. Once execution is complete, right-click again and choose View Table to confirm the data has loaded correctly.



Step 3. Generate Summary Statistics

Add a **Statistics** node to the canvas and connect it to the output port of the CSV Reader. Execute it and view the result. The output shows the mean, minimum, maximum, standard deviation, and sum for each numeric column. This step provides an immediate snapshot of data distribution and helps identify any anomalies before analysis begins

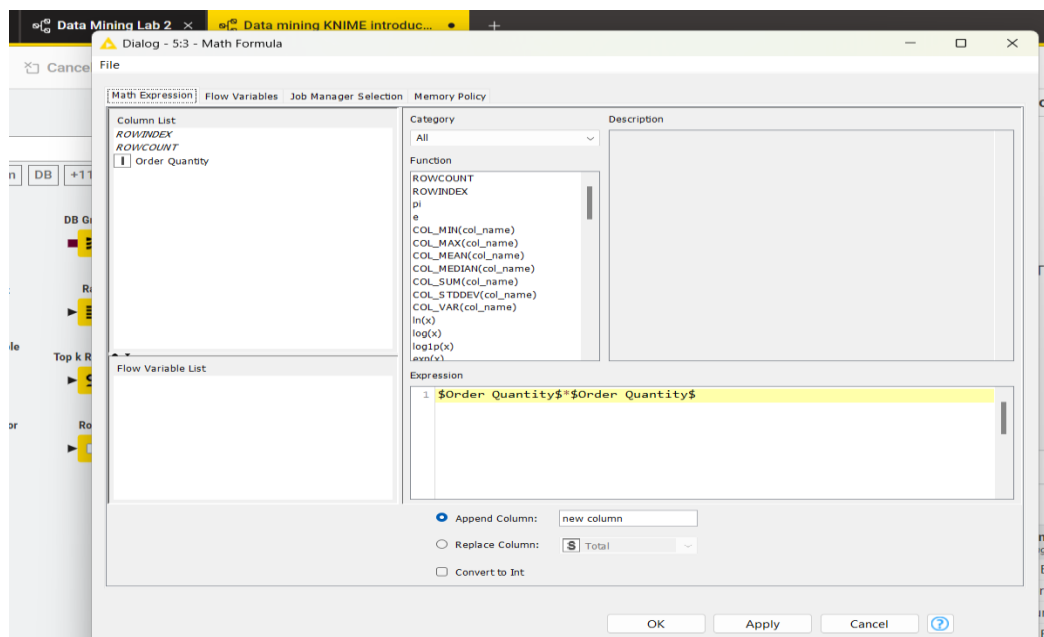


Step 5. Calculate the Revenue Column

Add a **Math Formula** node and connect it to the CSV Reader. Inside the node configuration, create a new column named Revenue using the formula:

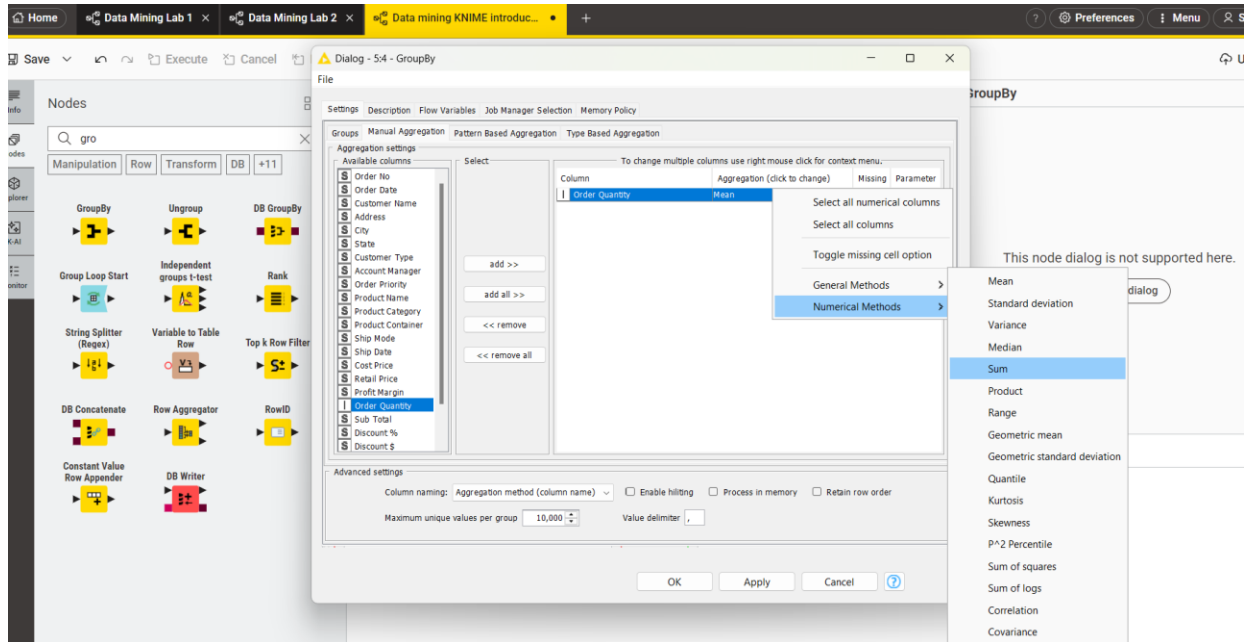
\$Quantity\$ * \$Price\$

Execute the node. Each row in the output now contains the revenue value for that individual transaction, which will serve as the base for all KPI calculations.



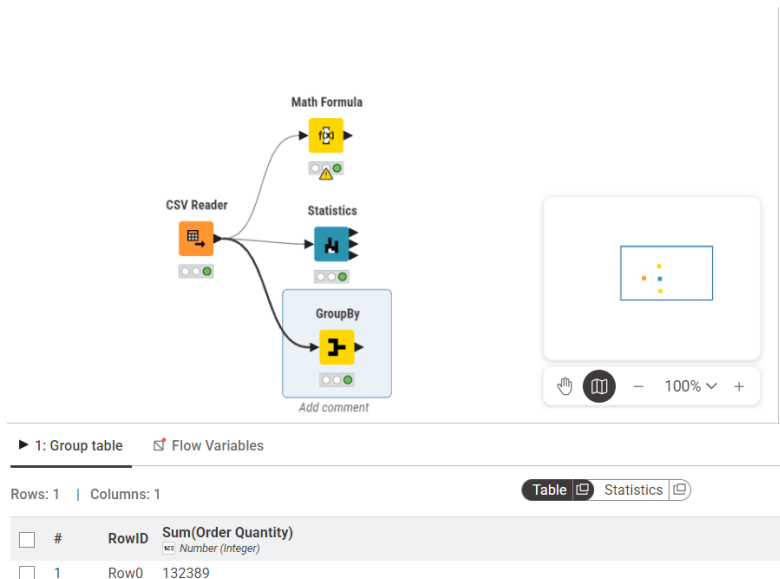
KPI 1 Total Revenue

Add a GroupBy node with no grouping column selected. In the Manual Aggregation tab, select the order quantity column and set the aggregation method to Sum. Execute the node. The single output row shows the total revenue across all transactions.



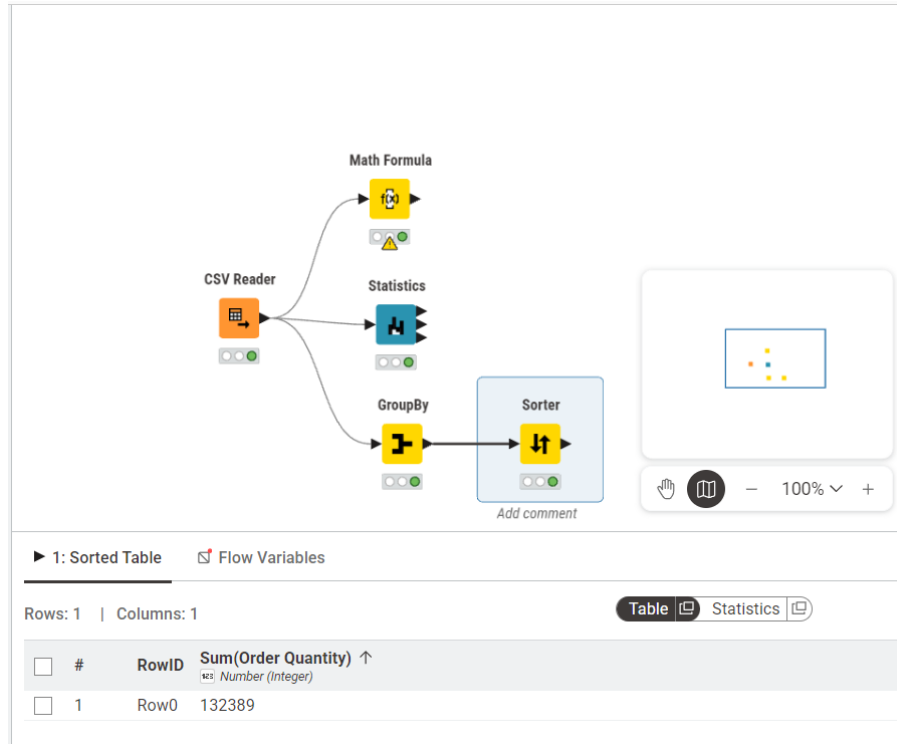
KPI 2 Average Order quantity

Add a GroupBy node and group by Order ID, aggregating Revenue using Sum. This produces the total revenue per order. Connect a second GroupBy node with no grouping column, aggregating the result using Mean. The output is the average revenue per order.



KPI 3 Category Revenue

Add a GroupBy node and group by the Category column, aggregating Revenue using Sum. Connect a Sorter node configured to sort by Revenue in descending order. The resulting table ranks each product category from highest to lowest earnings.



KPI Summary:

KPI	Method	KNIME Nodes Used
Total Revenue	Sum of (Quantity × Price)	Math Formula: GroupBy (Sum)
Average Order Value	Mean revenue per Order ID	GroupBy (Order ID) : GroupBy (Mean)
Category Revenue	Revenue grouped by Category	GroupBy (Category) : Sorter (Desc)
Top Products	Revenue grouped by Product Name	GroupBy (Product Name): Sorter (Desc)

Lab 2

CRISP-DM Methodology

Real World Problem

An e-commerce company is experiencing high customer churn and wants to predict which customers are likely to leave. The company has collected historical data on customer behaviour, purchase history, and service interactions. This lab applies the CRISP-DM (Cross-Industry Standard Process for Data Mining) framework to organise the full analytics lifecycle from business understanding through to deployment.

Dataset

Customer Churn Dataset containing features such as customer tenure, monthly charges, contract type, payment method, number of support tickets, and a binary churn label.

Practical Objectives:

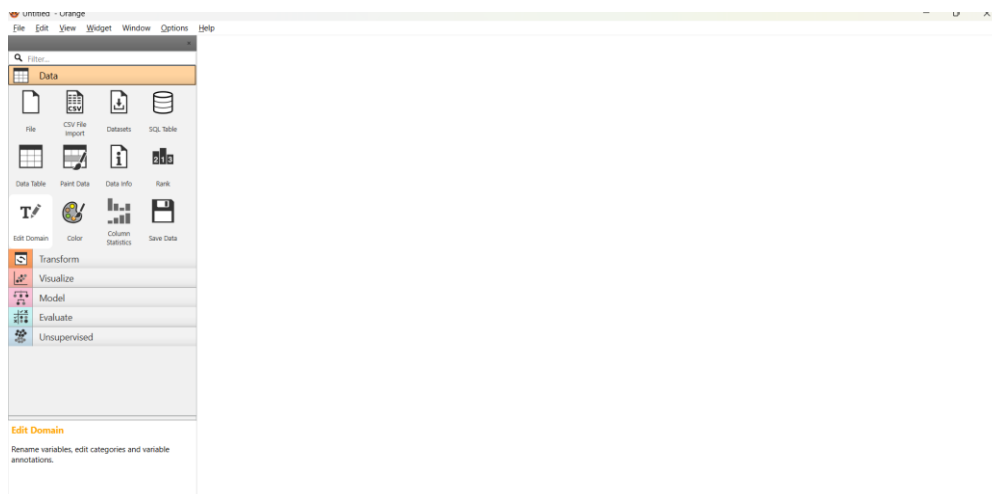
By the end of this lab, students will be able to:

- Define each phase of the CRISP-DM process and map it to a real business problem.
- Perform exploratory data analysis to understand churn patterns.
- Prepare and transform features suitable for predictive modelling.
- Build a basic classification model using Orange Data Mining.
- Evaluate the model using Churn Rate, Retention Rate, Accuracy, and ROC-AUC.

Procedure:

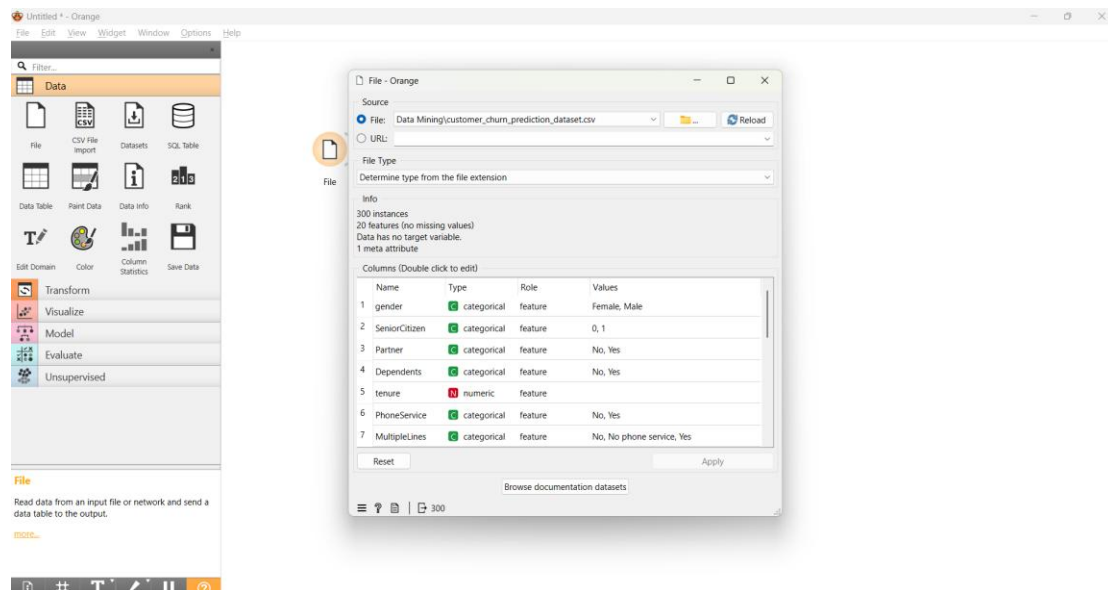
Step 1. Business Understanding

Open Orange Data Mining and create a new workflow. The business goal is to reduce churn by identifying at-risk customers. Define the mining goal: classify customers as Churn Yes or No. Note the key business KPIs: Churn Rate and Retention Rate.



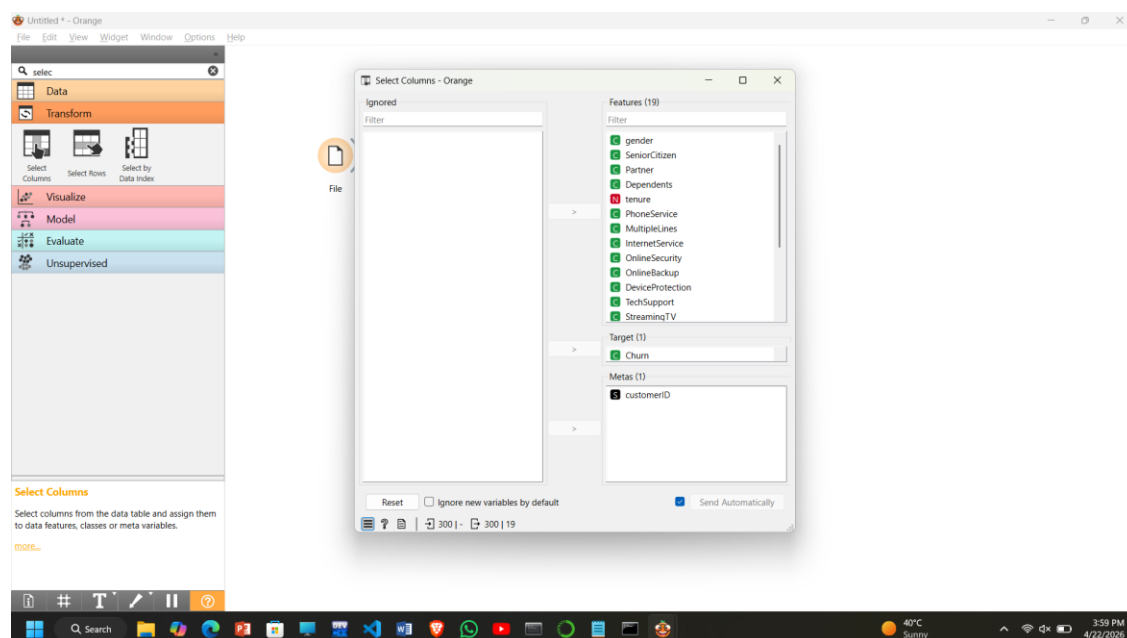
Step 2. Data Understanding

Add the File widget and load the Customer Churn CSV. Connect a Data Table widget to inspect records. Connect a Distributions widget to visualise the spread of numeric columns such as tenure and MonthlyCharges. Use the Feature Statistics widget to view missing counts and data types.



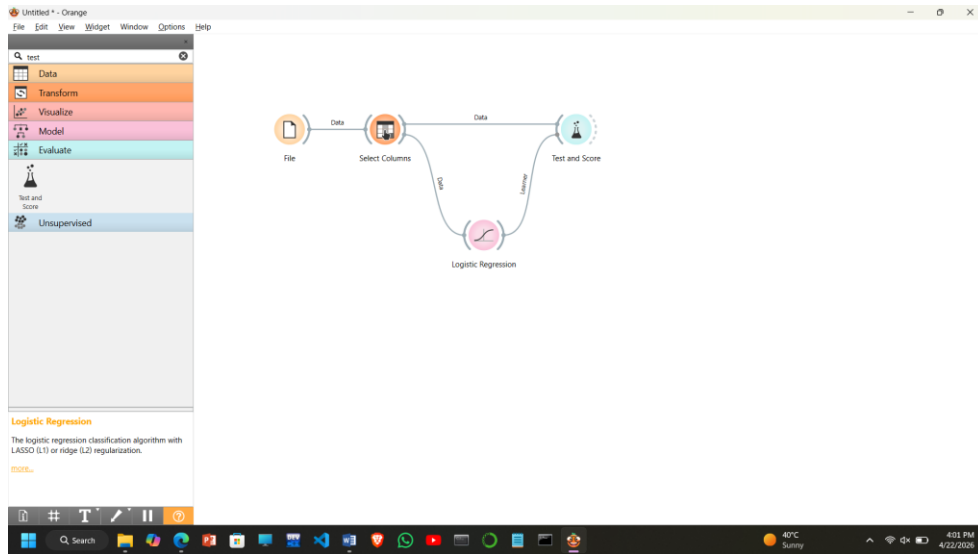
Step 3. Data Preparation

Add a Select Columns widget to remove irrelevant identifiers such as CustomerID. Connect an Impute widget to handle any missing values using the mean strategy for numeric columns. Add an Edit Domain widget to confirm the target variable Churn is set as the class attribute.



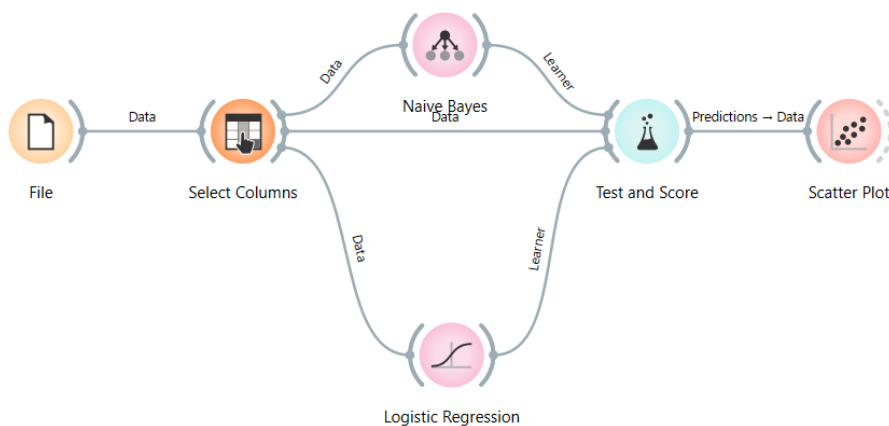
Step 4. Modelling

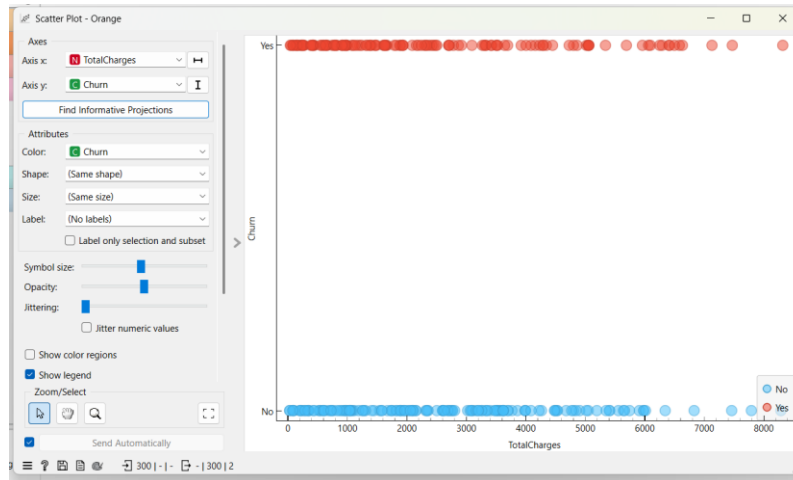
Add a Logistic Regression learner widget. Connect it to the preprocessed data. Also add a Naive Bayes learner for comparison. Connect both learners to a Test and Score widget configured to perform 10-fold cross-validation.



Step 5. Evaluation

View the Test and Score results panel. Record Accuracy, AUC, Precision, and Recall for each model. Connect an ROC Analysis widget to visualise classifier performance curves. A higher AUC indicates a better ability to separate churners from non-churners.





Test and Score - Orange

File Edit View Window Help

☒ Cross validation

Number of folds: 5

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 66 %

☒ Stratified

☐ Leave one out

☐ Test on train data

☐ Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
Logistic Regression	0.557	0.573	0.569	0.570	0.573	0.135
Naive Bayes	0.546	0.563	0.563	0.563	0.563	0.122

Compare models by: Area under ROC curve

☐ Negligible diff.: 0.1

	Logistic Reg...	Naive Bayes
Logistic Regression		0.580
Naive Bayes	0.420	

Table shows probabilities that the score for the model in the row is higher than that of the model in the column. Small numbers show the probability that the difference is negligible.

KPI Summary:

KPI	Method	Tool(s) Used
Churn Rate	% of customers labelled Churn = Yes	Orange: Distributions widget
Retention Rate	100% minus Churn Rate	Calculated from Data Table
Accuracy	Correct predictions / total predictions	Orange: Test and Score
ROC-AUC	Area under the ROC curve	Orange: ROC Analysis widget

Lab 3

Data Cleaning

Real World Problem

A hospital is preparing patient records collected from multiple departments for a disease prediction study. The raw data contains missing diagnostic values, duplicate patient entries from system migrations, and noisy sensor readings. Before any analysis can proceed, the data must be cleaned to a standard suitable for clinical machine learning.

Dataset

Healthcare Dataset patient records including age, gender, blood pressure, glucose, BMI, diagnosis codes, and admission dates, with intentionally introduced missing values, duplicates, and outliers.

Practical Objectives:

By the end of this lab, students will be able to:

- Identify and handle missing values using multiple imputation strategies.
- Detect and remove duplicate records from the dataset.
- Apply noise filtering using the IQR smoothing technique.
- Calculate a data quality score before and after cleaning using Python.

Procedure:

Step 1. Load and Profile the Data

Open Python and import the healthcare dataset using pandas: `df = pd.read_csv('healthcare_dataset.csv')`. Print `df.shape` to view the total row and column count. Use `df.dtypes` to inspect column types and `df.isnull().sum()` to identify which columns contain missing values. This profiling step determines the cleaning strategy to apply in the following steps.

Step 1 – Install & Import Libraries

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print('Libraries imported successfully.')
Libraries imported successfully.
```

Step 2 – Load and Profile the Data

```
[14]: # Load the dataset (replace the path with your actual file)
df = pd.read_csv('heart.csv')

print('Shape:', df.shape)
print('\nColumn Types:')
print(df.dtypes)
df.head()
```

Shape: (303, 14)

Column Types:

age	int64
sex	int64
cp	int64
trestbps	int64
chol	int64
fbs	int64
restecg	int64
thalach	int64
exang	int64
oldpeak	float64
slope	int64
ca	int64
thal	int64
target	int64
dtype:	object

```
[14]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

```

missing = df.isnull().sum()
missing_pct = (missing / len(df)) * 100

profile = pd.DataFrame({'Missing Count': missing, 'Missing %': missing_pct})
print('Missing Value Profile:')
print(profile[profile['Missing Count'] > 0])

```

```

Missing Value Profile:
Empty DataFrame
Columns: [Missing Count, Missing %]
Index: []

```

Step 2. Handle Missing Values

Separate the columns into numeric and categorical groups. For numeric columns such as BloodPressure, Glucose, and BMI, fill missing values using the column mean: `df[col].fillna(df[col].mean(), inplace=True)`. For categorical columns such as Gender and Diagnosis, fill missing values with the most frequent value: `df[col].fillna(df[col].mode()[0], inplace=True)`. Print `df.isnull().sum()` again to confirm all missing values have been filled.

Step 4 – Handle Missing Values

```

17]: df_clean = df.copy()

# Fill numeric columns with column mean
numeric_cols = df_clean.select_dtypes(include=[np.number]).columns
df_clean[numeric_cols] = df_clean[numeric_cols].fillna(df_clean[numeric_cols].mean())

# Fill categorical columns with most frequent value
cat_cols = df_clean.select_dtypes(include=['object']).columns
for col in cat_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])

print('Missing values after imputation:')
print(df_clean.isnull().sum())

Missing values after imputation:
age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       0
thal     0
target   0
dtype: int64

```

Step 3. Remove Duplicate Records

Record the row count before deduplication using `rows_before = len(df)`. Calculate the Duplicate Removal % as $(\text{rows_before} - \text{rows_after}) / \text{rows_before} \times 100$. Print the result to confirm how many duplicate records were removed from the dataset.

Step 5 – Remove Duplicate Records

```

18]: rows_before = len(df_clean)
df_clean = df_clean.drop_duplicates()
rows_after = len(df_clean)

duplicates_removed = rows_before - rows_after
dup_pct = (duplicates_removed / rows_before) * 100

print(f'Rows before duplicate removal : {rows_before}')
print(f'Rows after duplicate removal : {rows_after}')
print(f'Duplicates removed : {duplicates_removed} ({dup_pct:.2f}%)')

Rows before duplicate removal : 303
Rows after duplicate removal : 302
Duplicates removed : 1 (0.33%)

```


Step 4. Noise Filtering Using IQR

Apply the IQR method to detect and replace outliers in key numeric columns such as BloodPressure and Glucose. Calculate $Q1 = df[col].quantile(0.25)$ and $Q3 = df[col].quantile(0.75)$, then $IQR = Q3 - Q1$. Any value below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is considered an outlier. Replace these values with the column median. Print the outlier count per column before replacement to observe the extent of noise in the data.

▼ Step 6 – Outlier Detection and Removal (IQR Method)

```
[6]: def remove_outliers_iqr(dataframe, column):
      Q1 = dataframe[column].quantile(0.25)
      Q3 = dataframe[column].quantile(0.75)
      IQR = Q3 - Q1
      lower = Q1 - 1.5 * IQR
      upper = Q3 + 1.5 * IQR
      outliers = dataframe[(dataframe[column] < lower) | (dataframe[column] > upper)]
      print(f' {column}: {len(outliers)} outliers detected (range {lower:.2f} - {upper:.2f})')
      # Replace outliers with median
      median_val = dataframe[column].median()
      dataframe[column] = np.where(
          (dataframe[column] < lower) | (dataframe[column] > upper),
          median_val,
          dataframe[column]
      )
      return dataframe

target_cols = [c for c in ['testbps', 'chol', 'fbs', 'restecg'] if c in df_clean.columns]
print('Outlier detection:')
for col in target_cols:
    df_clean = remove_outliers_iqr(df_clean, col)

Outlier detection:
chol: 5 outliers detected (range 115.38 - 370.38)
fbs: 45 outliers detected (range 0.00 - 0.00)
```

Step 5. Calculate Data Quality Score

Compute the data quality score before and after cleaning using the formula: $\text{Quality Score} = (1 - \frac{\text{missing_cells}}{\text{total_cells}}) \times 100$. Use `df.isnull().sum().sum()` for the count of missing cells and `df.size` for the total cell count. Print both scores side by side to show the measurable improvement achieved through the cleaning steps. A score above 95% indicates the dataset is ready for downstream analysis.

Step 7 – Data Quality Score (After Cleaning)

```
[7]: score_after = data_quality_score(df_clean)
      print(f'Data Quality Score BEFORE cleaning : {score_before}%')
      print(f'Data Quality Score AFTER cleaning : {score_after}%')
      print(f'Improvement : +{score_after - score_before:.2f}%')

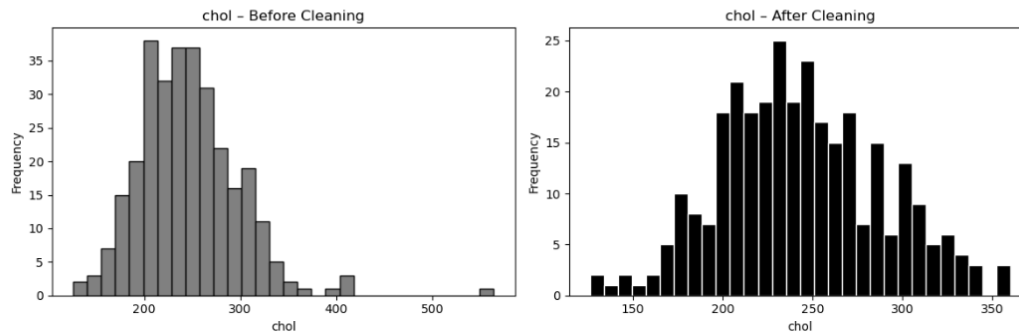
Data Quality Score BEFORE cleaning : 100.0%
Data Quality Score AFTER cleaning : 100.0%
Improvement : +0.00%
```

Step 8 – Visualise Before vs After (Distribution Plot)

```
[28]: if target_cols:
      col = target_cols[0]
      fig, axes = plt.subplots(1, 2, figsize=(12, 4))
      axes[0].hist(df[col].dropna(), bins=30, color='gray', edgecolor='black')
      axes[0].set_title(f'{col} - Before Cleaning')
      axes[0].set_xlabel(col)
      axes[0].set_ylabel('Frequency')

      axes[1].hist(df_clean[col], bins=30, color='black', edgecolor='white')
      axes[1].set_title(f'{col} - After Cleaning')
      axes[1].set_xlabel(col)
      axes[1].set_ylabel('Frequency')

      plt.tight_layout()
      plt.savefig('lab3_distribution.png', dpi=150)
      plt.show()
```



KPI Summary:

KPI Summary

```
[29]: kpi = pd.DataFrame([
      'KPI': ['% Missing Reduced', 'Duplicate Removal %', 'Data Quality Score (Before)', 'Data Quality Score (After)'],
      'Value': [
          f"round((df.isnull().sum().sum() / df.size) * 100, 2))% + 0%",
          f"{dup_pct:.2f}%",
          f"{score_before}%",
          f"{score_after}%"
      ]
  ])
print(kpi.to_string(index=False))
```

KPI	Value
% Missing Reduced	0.0% + 0%
Duplicate Removal %	0.33%
Data Quality Score (Before)	100.0%
Data Quality Score (After)	100.0%

KPI	Method	Tool(s) Used
% Missing Reduced	Missing cells before vs after imputation	Python: pandas fillna
Duplicate Removal %	Rows removed / original rows x 100	Python: drop_duplicates
Data Quality Score	(1 – missing ratio) x 100	Python: pandas
Outlier Count	Records outside 1.5 x IQR range per column	Python: IQR method

Lab 4

Data Preprocessing

Real World Problem

A bank needs to prepare loan application data for a risk prediction model. The raw data contains numeric columns on vastly different scales, categorical variables such as loan purpose, and skewed distributions. Proper preprocessing ensures the machine learning model trains effectively and produces unbiased predictions.

Dataset

Loan Prediction Dataset including features such as applicant income, co-applicant income, loan amount, loan term, credit history, education level, and employment status.

Practical Objectives:

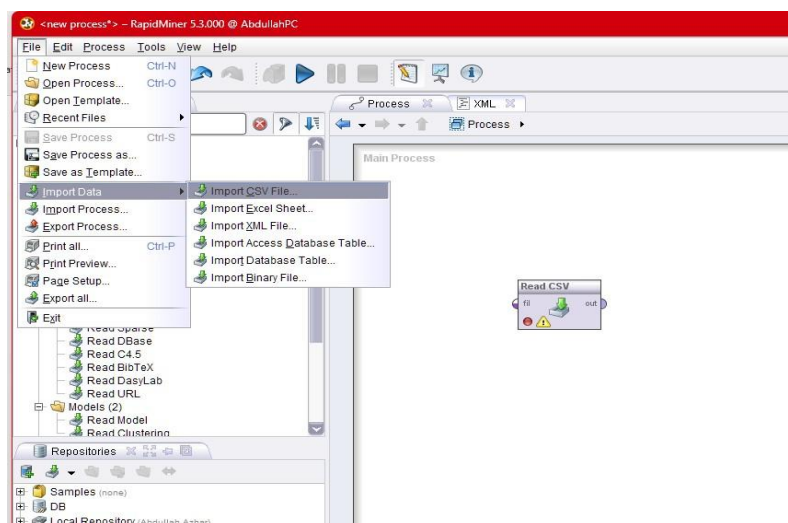
By the end of this lab, students will be able to:

- Apply Min-Max normalisation to scale numeric features to a 0 to 1 range.
- Apply Z-score standardisation to centre features around mean zero.
- Discretise a continuous variable such as loan amount into meaningful bins.
- Encode categorical variables using one-hot encoding.
- Compare data distribution before and after preprocessing using Python plots.

Procedure:

Step 1. Load the Dataset

In RapidMiner Studio, use the Read CSV operator to load the Loan Prediction dataset. Run the process and inspect the data. Identify numeric columns (ApplicantIncome, LoanAmount, Loan_Amount_Term) and categorical columns (Gender, Education, Property_Area).

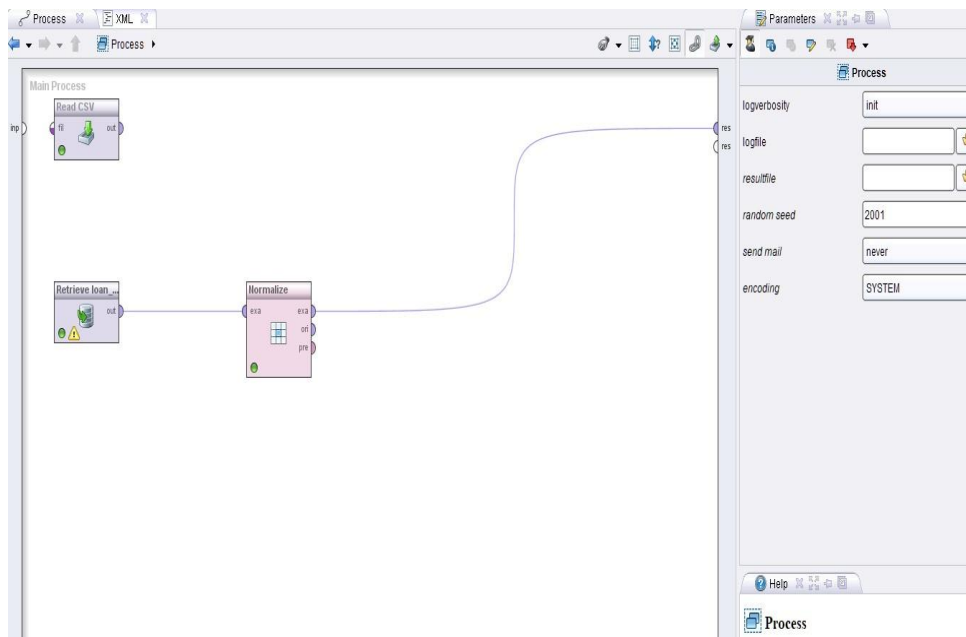


Step 2. Normalisation (Min-Max)

Add a Normalize operator. Set the method to range transformation with min = 0 and max = 1. Apply it to ApplicantIncome, CoapplicantIncome, and LoanAmount. Execute and verify that all selected columns now fall within the range 0 to 1.

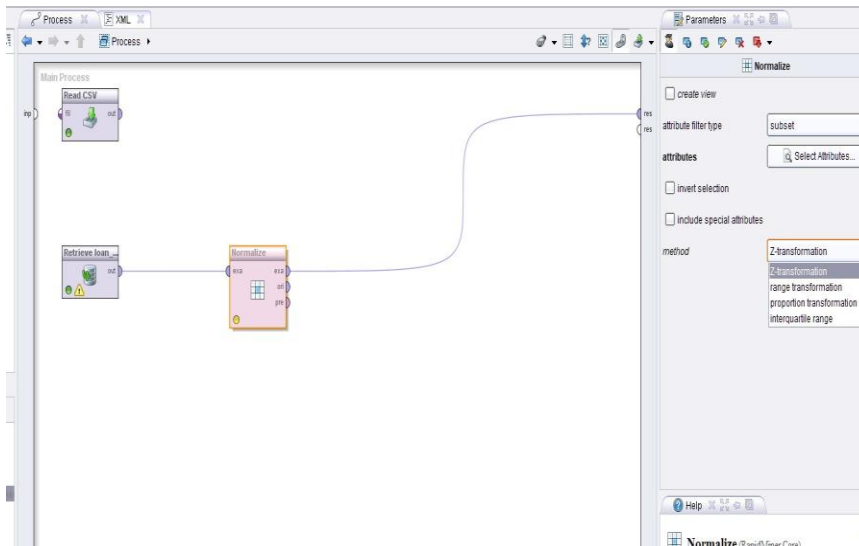


Row No.	loan_paid_...	annual_inc...	loan_amount
1	1	0.046	0.344
2	1	0.036	0.457
3	1	0.051	0.065
4	1	0.015	0.297
5	1	0.049	0.331
6	1	0.126	0.252
7	0	0.047	0.111
8	1	0.110	0.200



Step 3. Standardisation (Z-Score)

Add a second Normalize operator. Set the method to Z-transformation. Apply it to the same numeric columns. Compare the output with Min-Max: values will now be centred around 0 with standard deviation 1. Z-score is preferable when the data follows a Gaussian distribution.

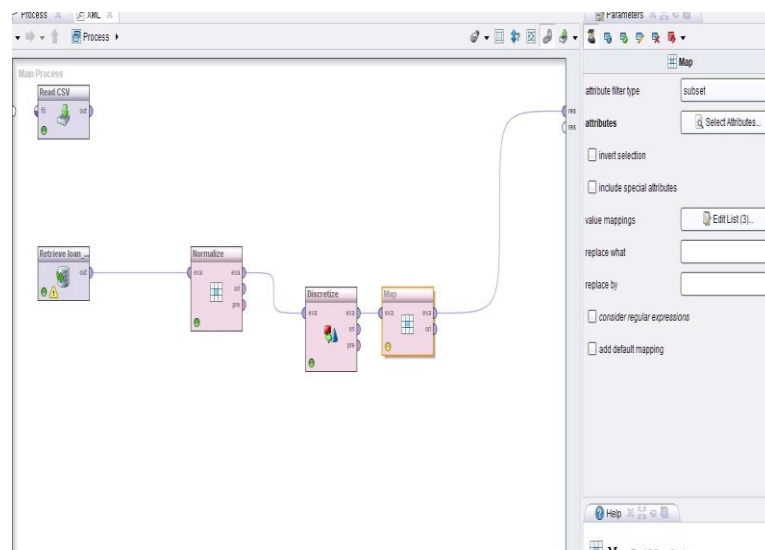


ExampleSet (20000 examples, 1 special attribute, 211 attributes)

Row No.	loan_paid_...	annual_inc...	loan_amount
1	1	-0.674	0.238
2	1	-0.815	0.876
3	1	-0.606	-1.336
4	1	-1.105	-0.022
5	1	-0.636	0.165
6	1	0.419	-0.280
7	0	-0.659	-1.072
8	1	0.316	0.001
9	1	0.676	-1.700

Step 4. Discretisation

Add a Discretize by Binning operator. Apply it to the LoanAmount column. Set the number of bins to 3 and label them Low, Medium, and High. Execute the process and verify the new categorical column in the results.



sampleSet (20000 examples, 1 special attribute, 21 regular attributes)

Row No.	loan_paid_off	annual_income	loan_amount	
	1	low	0.238	5%
	1	low	0.876	7%
	1	low	-1.336	4%
	1	low	-0.022	3%
	1	low	0.165	6%
	1	low	-0.280	2%
	0	low	-1.072	4%
	1	low	0.001	5%
	1	low	-1.700	3%

Step 5. One-Hot Encoding

Add a Nominal to Numerical operator. Select the Gender, Education, and Property_Area columns. Set the encoding type to Dummy Coding. Each category becomes a separate binary column. Execute and view the expanded dataset.

Step 6. Compare Distributions

In Python, plot histograms of ApplicantIncome before and after normalisation using matplotlib. Observe the change in scale. Calculate variance before and after to confirm the Variance Reduction KPI.

KPI Summary:

KPI	Method	Tool(s) Used
Variance Reduction	Variance before vs after normalisation	RapidMiner: Normalize
Distribution Improvement	Histogram shape before vs after	Python: matplotlib
Encoding Completeness	All categorical columns encoded	RapidMiner: Nominal to Numerical
Bin Coverage	% of records in each discretised bin	RapidMiner: Discretize

Lab 5

Market Basket Analysis

Real World Problem

A supermarket wants to develop a cross-selling strategy by identifying which products customers frequently buy together. By discovering strong association rules, the store can redesign shelf layouts, create product bundles, and target promotions. This lab manually calculates the core metrics and then validates them programmatically.

Dataset

Grocery Transactions Dataset (Instacart Market Basket Analysis) where each record represents a customer order containing a list of purchased product IDs.

Practical Objectives:

By the end of this lab, students will be able to: .

- Identify frequent itemsets from a transaction dataset.
- Interpret association rules in a business context.
- Validate manual calculations using Python's mlxtend library.

Procedure:

Step 1&2. Load Data in Python

Import the dataset using pandas. Group orders by order_id and aggregate product names into lists. Convert to a list-of-lists format required by mlxtend: transactions = df.groupby('order_id')['product_name'].apply(list).tolist()

```
Step 1 – Install & Import Libraries

[4] ✓ 0s
import warnings
warnings.filterwarnings('ignore')

[5] ✓ 0s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

print('libraries imported successfully.')

libraries imported successfully.
```

```
Step 2 – Load the Full Dataset

df = pd.read_csv('/content/Groceries_dataset.csv')
transactions = df.groupby('Member_number')['itemDescription'].apply(list).tolist()

print(f'Total transactions loaded: {len(transactions)}')
print(f'Sample transaction: {transactions[0]}')

*** Total transactions loaded: 3898
Sample transaction: ['soda', 'canned beer', 'sausage', 'sausage', 'whole milk', 'wh...
```

Step 3. One-Hot Encode Transactions

Use mlxtend's TransactionEncoder to convert the list-of-lists into a binary DataFrame. Each column represents a product and each row a transaction. Products present in the order are marked True. Fit and transform the transaction data.

```
Step 3 – One-Hot Encode Transactions
```

```
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_encoded = pd.DataFrame(te_array, columns=te.columns_)

print('Encoded shape:', df_encoded.shape)
df_encoded.head()
```

Encoded shape: (3898, 167)

	Instant food products	UHT- milk	abrasive cleaner	artif. sweetener	baby cosmetics	bags	baking powder	bathroom cleaner	beef	be
0	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	True	False
2	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False

5 rows x 167 columns

Step 4. Find Frequent Itemsets

Apply the apriori function from mlxtend.frequent_patterns with min_support = 0.01. This returns all itemsets appearing in at least 1% of orders. Sort results by support in descending order and inspect the top 10 frequent itemsets.

```
Step 4 – Find Frequent Itemsets
```

```
+ Code + Text
```

```
frequent_itemsets = apriori(df_encoded, min_support=0.01, use_colnames=True)
frequent_itemsets = frequent_itemsets.sort_values('support', ascending=False)

print(f'Total frequent itemsets found: {len(frequent_itemsets)}')
print('\nTop 10 Frequent Itemsets:')
print(frequent_itemsets.head(10).to_string(index=False))
```

Total frequent itemsets found: 3016

Top 10 Frequent Itemsets:

support	itemsets
0.458184	(whole milk)
0.376603	(other vegetables)
0.349666	(rolls/buns)
0.313494	(soda)
0.282966	(yogurt)
0.233710	(tropical fruit)
0.230631	(root vegetables)
0.213699	(bottled water)
0.206003	(sausage)
0.191380	(other vegetables, whole milk)

Step 5. Generate and Interpret Rules

Apply the `association_rules` function with `metric = 'lift'` and `min_threshold = 1.5`. Display the top rules sorted by lift. Identify the top 5 rules with the highest lift and explain what each means for the supermarket's cross-sell strategy.

Step 5 – Generate Association Rules

```
rules = association_rules(frequent_itemsets, metric='lift', min_threshold=1.5)
rules = rules.sort_values('lift', ascending=False)

print(f'Total rules generated: {len(rules)}')
print('\nTop 5 Rules by Lift:')
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head(5).to_string())
```

*** Total rules generated: 15260

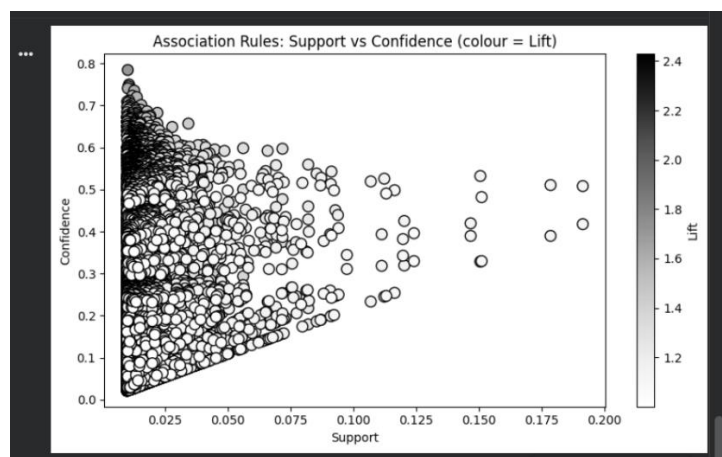
Top 5 Rules by Lift:

antecedents	consequents	support	confidence	lift
(yogurt, rolls/buns)	(sausage, other vegetables, whole milk)	0.00	0.00	0.00
(sausage, other vegetables, whole milk)	(yogurt, rolls/buns)	0.00	0.00	0.00
(yogurt, rolls/buns, other vegetables)	(sausage, whole milk)	0.00	0.00	0.00
(sausage, whole milk)	(yogurt, rolls/buns, other vegetables)	0.00	0.00	0.00
(curd, yogurt)	(sausage, whole milk)	0.00	0.00	0.00

Step 6. Visualize Support VS Confidence

Step 6 – Visualise Support vs Confidence

```
if len(rules) > 0:
    plt.figure(figsize=(8, 5))
    scatter = plt.scatter(
        rules['support'],
        rules['confidence'],
        c=rules['lift'],
        cmap='Greys',
        edgecolors='black',
        s=80
    )
    plt.colorbar(scatter, label='Lift')
    plt.xlabel('Support')
    plt.ylabel('Confidence')
    plt.title('Association Rules: Support vs Confidence (colour = Lift)')
    plt.tight_layout()
    plt.savefig('lab5_rules_scatter.png', dpi=150)
    plt.show()
    # print('Plot saved as lab5_rules_scatter.png')
    # plt.plot()
```



KPI Summary:

```
KPI Summary

high_lift_rules = rules[rules['lift'] > 1.5]

kpi = pd.DataFrame({
    'KPI': ['Frequent Itemsets Found', 'Total Rules Generated', 'High Lift Rules (Lift > 1.5)', 'Top Lift Value'],
    'Value': [
        len(frequent_itemsets),
        len(rules),
        len(high_lift_rules),
        f"{rules['lift'].max():.4f}" if len(rules) > 0 else 'N/A'
    ]
})
print(kpi.to_string(index=False))
```

KPI	Value
Frequent Itemsets Found	3016
Total Rules Generated	15260
High Lift Rules (Lift > 1.5)	2376
Top Lift Value	2.4287

KPI	Method	Tool(s) Used
High Lift Rules	Rules with Lift greater than 1.5 identified	Python: mlxtend association_rules
Frequent Itemsets	Itemsets meeting min_support threshold	Python: mlxtend apriori
Cross-Sell Potential	Top 5 rules ranked by Lift	Manual calculation + Python
Rule Count	Total rules generated at threshold	Python: association_rules

Lab 6

Apriori Algorithm

Real World Problem

A retail chain wants to identify product combinations that are frequently purchased together across all store branches. The management wants an algorithmic approach that systematically discovers all association rules. This lab implements the Apriori algorithm using Python to explore how the support threshold affects the number of rules generated.

Dataset

Transaction Dataset where each row represents one transaction and columns represent products encoded as binary values (1 = purchased, 0 = not purchased).

Practical Objectives:

By the end of this lab, students will be able to:

- Explain the Apriori algorithm's candidate-generation and pruning steps.
- Validate results in Python using the mlxtend library.
- Analyse how changing the support threshold impacts the number of rules generated.

Procedure:

Step 1. Understand the Apriori Principle

The Apriori principle states: if an itemset is infrequent, all its supersets are also infrequent. This property prunes the search space. The algorithm proceeds in passes: first finding all frequent 1-itemsets, then 2-itemsets, and so on, until no more frequent itemsets are found.

Step 2 – Apriori Principle (Explanation)

If an itemset is infrequent, ALL its supersets are also infrequent.
This prunes the candidate space and avoids checking every possible combination.

Step 2. Implement Apriori in Python

Load the transaction dataset in Python. Apply one-hot encoding using TransactionEncoder. Run the apriori function with min_support = 0.05. Generate rules using association_rules with metric = 'confidence' and min_threshold = 0.3. Print the top 10 rules sorted by lift.

Step 1 – Install & Import Libraries

```
# pip install mlxtend

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import time
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

print('libraries imported successfully.')

libraries imported successfully.
```

```

Step 3 – Load the Dataset

df = pd.read_csv('/content/basket_analysis.csv')

if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])

print(f'Total transactions: {len(df)}')
print(f'Encoded shape: ', df.shape)
df_encoded = df
df_encoded.head()

```

Total transactions: 999
 Encoded shape: (999, 16)

	Apple	Bread	Butter	Cheese	Corn	Dill	Eggs	Ice cream	Kidney Beans	Milk	Garlic	Onion	Sugar	Broccoli	Yogurt
0	False	True	False	False	True	True	False	True	False	False	False	False	True	False	True
1	False	False	False	False	False	False	False	False	True	False	False	False	False	False	False
2	True	False	True	False	False	True	False	True	False	True	False	False	False	False	True
3	False	False	True	True	False	True	False	False	False	True	True	True	False	False	False
4	True	True	False	False	False	False	False	False	False	False	False	False	False	False	False

Step 3. Running Apriori and Rule generation

```

Step 4 – Run Apriori and Generate Rules

MIN_SUPPORT = 0.05
MIN_CONFIDENCE = 0.3

start = time.time()
frequent_itemsets = apriori(df_encoded, min_support=MIN_SUPPORT, use_colnames=True)
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=MIN_CONFIDENCE)
elapsed = time.time() - start

rules = rules.sort_values('lift', ascending=False)

print(f'Frequent itemsets : {len(frequent_itemsets)}')
print(f'Rules generated : {len(rules)}')
print(f'Execution time : {elapsed:.4f} seconds')
print(f'Top 10 Rules by Lift:')
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head(10).to_string(index=False))

```

Frequent itemsets : 947
 Rules generated : 3883
 Execution time : 0.0639 seconds

antecedents	consequents	support	confidence	lift
(Dill, Cheese)	(Onion, Kidney Beans)	0.055055	0.318734	1.826022
(Onion, Kidney Beans)	(Dill, Cheese)	0.055055	0.323529	1.826022
(Dill, Broccoli)	(Onion, Chocolate)	0.060060	0.357143	1.820335
(Onion, Chocolate)	(Dill, Broccoli)	0.060060	0.306122	1.820335
(Dill, Kidney Beans)	(Chocolate, Cheese)	0.057057	0.331395	1.779914
(Chocolate, Cheese)	(Dill, Kidney Beans)	0.057057	0.306452	1.779914
(Apple, Broccoli)	(Sugar, Corn)	0.055055	0.331325	1.770021
(Bread, Kidney Beans)	(Milk, Corn)	0.057057	0.341317	1.766715
(Chocolate, Broccoli)	(Milk, Dill)	0.062062	0.333333	1.752632
(Milk, Dill)	(Chocolate, Broccoli)	0.062062	0.326316	1.752632

Step 5. Threshold Impact Analysis

Run the Apriori algorithm in Python three times with different support thresholds: 0.10, 0.05, and 0.01. Record the number of frequent itemsets and rules generated at each threshold. Create a comparison showing how lowering the threshold increases rule count but reduces overall quality.

Step 5 – Threshold Impact Analysis

Run Apriori at three different support levels and compare the results.

```
thresholds = [0.10, 0.05, 0.01]
results = []

for thresh in thresholds:
    t0 = time.time()
    fi = apriori(df_encoded, min_support=thresh, use_colnames=True)
    if len(fi) > 0:
        r = association_rules(fi, metric='confidence', min_threshold=0.3)
    else:
        r = pd.DataFrame()
    t1 = time.time()
    results.append({
        'Support Threshold': thresh,
        'Frequent Itemsets': len(fi),
        'Rules Generated': len(r),
        'Execution Time(s)': round(t1 - t0, 4)
    })

impact_df = pd.DataFrame(results)
print('Threshold Impact Analysis:')
print(impact_df.to_string(index=False))
```

Support Threshold	Frequent Itemsets	Rules Generated	Execution Time(s)
0.10	169	339	0.0244
0.05	947	3083	0.0497
0.01	12224	97636	2.3720

Step 6. Visual Threshold Impact

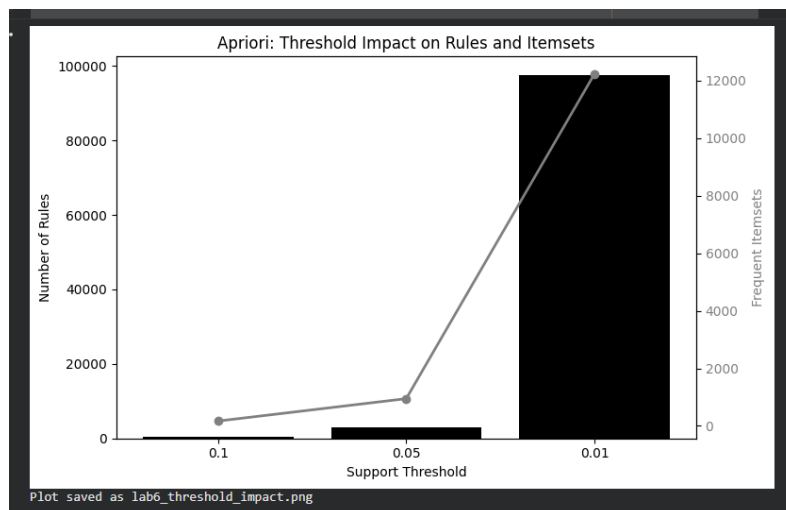
Step 6 – Visualise Threshold Impact

```
fig, ax1 = plt.subplots(figsize=(8, 5))

ax1.bar([str(t) for t in impact_df['Support Threshold']],
        impact_df['Rules Generated'], color='black', label='Rules Generated')
ax1.set_xlabel('Support Threshold')
ax1.set_ylabel('Number of Rules', color='black')
ax1.tick_params(axis='y', labelcolor='black')

ax2 = ax1.twinx()
ax2.plot([str(t) for t in impact_df['Support Threshold']],
         impact_df['Frequent Itemsets'], color='gray', marker='o', linewidth=2, label='Frequent Itemsets')
ax2.set_ylabel('Frequent Itemsets', color='gray')
ax2.tick_params(axis='y', labelcolor='gray')

plt.title('Apriori: Threshold Impact on Rules and Itemsets')
fig.tight_layout()
plt.savefig('lab6_threshold_impact.png', dpi=150)
plt.show()
print('Plot saved as lab6_threshold_impact.png')
```



KPI Summary:

```
KPI Summary

print('KPI Summary (at min_support=0.05, min_confidence=0.3):')
kpi = pd.DataFrame({
    'KPI': ['Frequent Itemsets', 'Rules Generated', 'Highest Lift', 'Execution Time (s)'],
    'Value': [
        len(frequent_itemsets),
        len(rules),
        f"{rules['lift'].max():.4f}" if len(rules) > 0 else 'N/A',
        f'{elapsed:.4f}'
    ]
})
print(kpi.to_string(index=False))

KPI Summary (at min_support=0.05, min_confidence=0.3):
KPI      Value
Frequent Itemsets    947
Rules Generated      3083
Highest Lift         1.8260
Execution Time (s)   0.0639
```

KPI	Method	Tool(s) Used
Number of Rules	Total association rules generated	KNIME: Apriori node + Python
Support Threshold Impact	Rule count at 0.10 / 0.05 / 0.01	Python: mlxtend apriori
Highest Lift Rule	Maximum lift value among all rules	Both tools
Execution Comparison	KNIME vs Python rule agreement	Manual verification

Lab 7

FP-Growth Algorithm

Real World Problem

A large retail chain processes millions of transactions daily. The Apriori algorithm becomes too slow on such datasets due to repeated database scans. The FP-Growth algorithm addresses this by compressing the database into an FP-Tree structure, enabling much faster frequent pattern mining. This lab constructs an FP-Tree manually and then validates performance on a large dataset.

Dataset

Large Retail Dataset containing thousands of grocery or retail orders, enabling a meaningful comparison of execution time between Apriori and FP-Growth.

Practical Objectives:

By the end of this lab, students will be able to:

- Construct an FP-Tree manually from a small set of sample transactions.
- Explain how FP-Growth avoids repeated database scans.
- Implement FP-Growth in Python using the mlxtend library.
- Use Orange Data Mining to visualise frequent patterns.
- Compare FP-Growth execution time and rule count against Apriori.

Procedure:

Step 1. Manual FP-Tree Construction

Take 6 sample transactions. Sort items in each transaction by their global frequency with the most frequent item first. Build the FP-Tree by inserting transactions one by one, sharing common prefixes. Draw the resulting tree structure. Identify the conditional pattern bases for the least frequent item and extract frequent patterns from them.

Step 1 – Install & Import Libraries

```
# !pip install mlxtend

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import time
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, apriori, association_rules

print('Libraries imported successfully.')

Libraries imported successfully.
```

Step 2 – Manual FP-Tree Concept (Small Sample)

We trace the FP-Tree construction step by step on 6 transactions.

```
manual_transactions = [
    ['Milk', 'Bread', 'Butter', 'Eggs'],
    ['Milk', 'Bread', 'Butter'],
    ['Milk', 'Bread'],
    ['Milk', 'Eggs'],
    ['Bread', 'Butter'],
    ['Bread', 'Eggs'],
]

from collections import Counter
item_freq = Counter(item for t in manual_transactions for item in t)
print('Global Item Frequencies:')
for item, freq in sorted(item_freq.items(), key=lambda x: -x[1]):
    print(f' {item}: {freq}')

print('\nSorted Transactions (most frequent item first):')
for i, t in enumerate(manual_transactions):
    sorted_t = sorted(t, key=lambda x: -item_freq[x])
    print(f' T{i+1}: {sorted_t}')

print('\n[FP-Tree construction traced above - draw the tree from these sorted transactions]')
print('Root -> Milk(4) -> Bread(3) -> Butter(2)')
print('          -> Eggs(1)')
print('          -> Eggs(1)')
print('          -> Eggs(1)')
print('          -> Bread(2) -> Butter(1)')
print('          -> Eggs(1)')
```

```
print('\n[FP-Tree construction traced above - draw the tree from these sorted transactions]')
print('Root -> Milk(4) -> Bread(3) -> Butter(2)')
print('          -> Eggs(1)')
print('          -> Eggs(1)')
print('          -> Eggs(1)')
print('          -> Bread(2) -> Butter(1)')
print('          -> Eggs(1)')

--- Global Item Frequencies:
    Bread: 5
    Milk: 4
    Butter: 3
    Eggs: 3

Sorted Transactions (most frequent item first):
T1: ['Bread', 'Milk', 'Butter', 'Eggs']
T2: ['Bread', 'Milk', 'Butter']
T3: ['Bread', 'Milk']
T4: ['Milk', 'Eggs']
T5: ['Bread', 'Butter']
T6: ['Bread', 'Eggs']

[FP-Tree construction traced above - draw the tree from these sorted transactions]
Root -> Milk(4) -> Bread(3) -> Butter(2)
          -> Eggs(1)
          -> Eggs(1)
          -> Eggs(1)
          -> Bread(2) -> Butter(1)
          -> Eggs(1)
```

Step 2. Implement FP-Growth in Python

Load the retail dataset in Python. Apply TransactionEncoder to produce a binary DataFrame. Import time and record the start time. Run fpgrowth from mlxtend.frequent_patterns with min_support = 0.01. Record the end time and calculate total execution time. Generate association rules from the result.

Step 3 – Load Dataset and Encode

```
import pandas as pd
import numpy as np

df_raw = pd.read_csv('/content/sample_data/retail_sales_dataset.csv')

df_raw.columns = df_raw.columns.str.strip()

df_raw = df_raw.dropna(subset=['Product Category', 'Transaction ID'])

df_raw['Transaction ID'] = df_raw['Transaction ID'].astype(str)

transactions = df_raw.groupby('Transaction ID')['Product Category'].apply(list).tolist()

te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_encoded = pd.DataFrame(te_array, columns=te.columns_)

print(f'Total transactions : {len(transactions)}')
print(f'Unique items      : {len(te.columns_)}')
print(f'Encoded shape       : {df_encoded.shape}')

--- Total transactions : 1000
    Unique items      : 3
    Encoded shape     : (1000, 3)
```


Step 4 – Run FP-Growth

```
MIN_SUPPORT = 0.01

start_fp = time.time()
fi_fp = fpgrowth(df_encoded, min_support=MIN_SUPPORT, use_colnames=True)
rules_fp = association_rules(fi_fp, metric='lift', min_threshold=1.0)
elapsed_fp = time.time() - start_fp

rules_fp = rules_fp.sort_values('lift', ascending=False)
print(f'FP-Growth Results (min_support={MIN_SUPPORT}):')
print(f' Frequent itemsets : {len(fi_fp)}')
print(f' Rules generated   : {len(rules_fp)}')
print(f' Execution time     : {elapsed_fp:.4f} seconds')

FP-Growth Results (min_support=0.01):
Frequent itemsets : 3
Rules generated   : 0
Execution time     : 0.0108 seconds
```

Step 5 – Run Apriori (Same Dataset, Same Threshold)

```
start_ap = time.time()
fi_ap = apriori(df_encoded, min_support=MIN_SUPPORT, use_colnames=True)
rules_ap = association_rules(fi_ap, metric='lift', min_threshold=1.0)
elapsed_ap = time.time() - start_ap

print(f'Apriori Results (min_support={MIN_SUPPORT}):')
print(f' Frequent itemsets : {len(fi_ap)}')
print(f' Rules generated   : {len(rules_ap)}')
print(f' Execution time     : {elapsed_ap:.4f} seconds')

Apriori Results (min_support=0.01):
Frequent itemsets : 3
Rules generated   : 0
Execution time     : 0.0101 seconds
```

Step 3. Compare with Apriori

Run the apriori function on the same dataset with the same min_support = 0.01. Record execution time for both algorithms. Compare total rules generated and execution time. FP-Growth should be significantly faster because it scans the database only twice.

Step 6 – Compare FP-Growth vs Apriori

```
comparison = pd.DataFrame({
    'Algorithm'      : ['FP-Growth', 'Apriori'],
    'Frequent Itemsets' : [len(fi_fp), len(fi_ap)],
    'Rules Generated'  : [len(rules_fp), len(rules_ap)],
    'Time (seconds)'   : [round(elapsed_fp, 4), round(elapsed_ap, 4)]
})
print('Algorithm Comparison:')
print(comparison.to_string(index=False))

speedup = elapsed_ap / elapsed_fp if elapsed_fp > 0 else float('inf')
print(f'\nFP-Growth is {speedup:.2f}x faster than Apriori on this dataset.')

Algorithm Comparison:
Algorithm Frequent Itemsets Rules Generated Time (seconds)
FP-Growth 3 0 0.0108
Apriori 3 0 0.0101

FP-Growth is 0.94x faster than Apriori on this dataset.
```

Step 4. Visualising the FP-growth

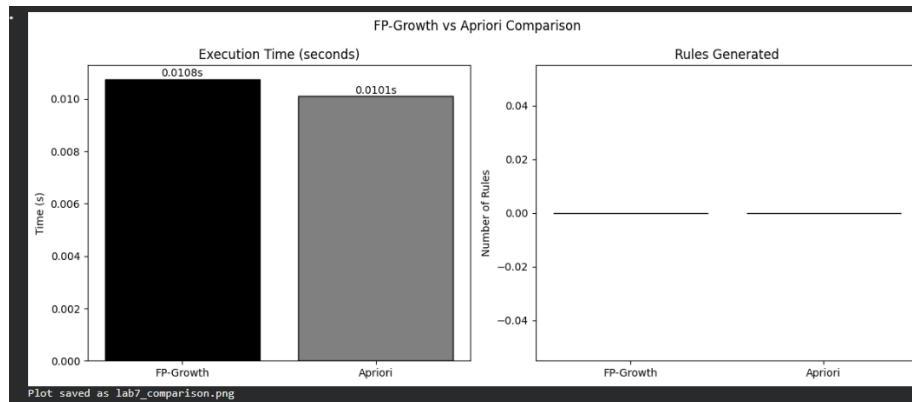
Step 7 – Visualise Execution Time Comparison

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].bar(['FP-Growth', 'Apriori'],
            [elapsed_fp, elapsed_ap],
            color=['black', 'gray'], edgecolor='black')
axes[0].set_title('Execution Time (seconds)')
axes[0].set_ylabel('Time (s)')
for i, v in enumerate([elapsed_fp, elapsed_ap]):
    axes[0].text(i, v + 0.0001, f'{v:.4f}s', ha='center', fontsize=10)

axes[1].bar(['FP-Growth', 'Apriori'],
            [len(rules_fp), len(rules_ap)],
            color=['black', 'gray'], edgecolor='black')
axes[1].set_title('Rules Generated')
axes[1].set_ylabel('Number of Rules')

plt.suptitle('FP-Growth vs Apriori Comparison')
plt.tight_layout()
plt.savefig('lab7_comparison.png', dpi=150)
plt.show()
print('Plot saved as lab7_comparison.png')
```



Step 5. Interpret Results

Identify the top 5 rules by lift from the FP-Growth output. For each rule, explain the business meaning: which products are bought together, and what cross-sell or placement strategy could be derived

Step 8 – Top 5 FP-Growth Rules (Business Interpretation)

```
print('Top 5 Rules by Lift (FP-Growth):')
top5 = rules_fp[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head(5)
print(top5.to_string(index=False))

print('\nBusiness Interpretation:')
for i, row in top5.iterrows():
    ant = ', '.join(list(row['antecedents']))
    cons = ', '.join(list(row['consequents']))
    print(f' Rule: {ant} -> {cons} | Lift: {row["lift"]:.3f}')
    print(f' Meaning: Customers who buy [{ant}] are {row["lift"]:.1f}x more likely to also buy [{cons}].')
    print()
```

```
Top 5 Rules by Lift (FP-Growth):
Empty DataFrame
Columns: [antecedents, consequents, support, confidence, lift]
Index: []
```

```
Business Interpretation:
```

KPI Summary:

```
KPI Summary

❏ kpi = pd.DataFrame({
    'KPI': [
        'FP-Growth Execution Time',
        'Apriori Execution Time',
        'Speed Improvement',
        'FP-Growth Rules',
        'Apriori Rules',
        'Top Lift Value (FP-Growth)'
    ],
    'Value': [
        f'{elapsed_fp:.4f} s',
        f'{elapsed_ap:.4f} s',
        f'{speedup:.2f}x faster',
        len(rules_fp),
        len(rules_ap),
        f"{rules_fp['lift'].max():.4f}" if len(rules_fp) > 0 else 'N/A'
    ]
})
print(kpi.to_string(index=False))

***
          KPI      Value
FP-Growth Execution Time  0.0140 s
Apriori Execution Time    0.0190 s
Speed Improvement      1.36x faster
FP-Growth Rules          0
Apriori Rules            0
Top Lift Value (FP-Growth)  N/A
```

KPI	Method	Tool(s) Used
Execution Time	FP-Growth time vs Apriori time (seconds)	Python: time module
Rule Count	Total rules generated at same threshold	Python: mlxtend / Orange
FP-Tree Depth	Number of levels in manually constructed tree	Manual construction
Top Lift Rule	Highest lift association rule found	Python + Orange

Lab 8

Python Basics for Data Processing

Real World Problem

A data analyst needs to build a reusable Python module that can be applied to any structured dataset to perform standard data processing tasks. Instead of writing repetitive code for each project, the analyst wants a class-based solution with methods for loading, inspecting, cleaning, and summarising data.

Dataset

Any structured CSV dataset. For demonstration, the Retail Sales or Loan Prediction dataset from earlier labs can be reused.

Practical Objectives:

By the end of this lab, students will be able to:

- Create and manipulate Python lists and dictionaries for data representation.
- Define a Python class with initialisation and processing methods.
- Implement methods for loading, inspecting, cleaning, and summarising data.
- Apply the class to a real dataset and verify output correctness.
- Evaluate code efficiency and reusability across different datasets.

Procedure:

Step 1. Python Lists and Dictionaries Review

Create a list of column names from the chosen dataset. Create a dictionary mapping each column name to its data type, for example {'Age': 'int', 'Income': 'float', 'Category': 'str'}. Iterate over the dictionary using a for loop and print each column name and its type. Demonstrate list slicing, appending, and dictionary key lookup.

Step 1 – Python Lists and Dictionaries Review

```
# ----- Lists -----
columns = ['Age', 'Income', 'LoanAmount', 'CreditHistory', 'Status']
print('All columns      : ', columns)
print('First 3 columns  : ', columns[:3])
print('Last column     : ', columns[-1])

columns.append('LoanTerm')
print('After append    : ', columns)

columns.remove('LoanTerm')
print('After remove    : ', columns)

*** All columns      : ['Age', 'Income', 'LoanAmount', 'CreditHistory', 'Status']
    First 3 columns  : ['Age', 'Income', 'LoanAmount']
    Last column     : Status
    After append    : ['Age', 'Income', 'LoanAmount', 'CreditHistory', 'Status', 'LoanTerm']
    After remove    : ['Age', 'Income', 'LoanAmount', 'CreditHistory', 'Status']
```

```

# ----- Dictionaries -----
column_types = {
    'Age'       : 'int',
    'Income'    : 'float',
    'LoanAmount': 'float',
    'CreditHistory' : 'int',
    'Status'    : 'str'
}

print('Column Type Mapping:')
for col, dtype in column_types.items():
    print(f' {col:<20} -> {dtype}')

print('\nLookup Income type :', column_types['Income'])

column_types['Gender'] = 'str'
print('Keys after adding Gender:', list(column_types.keys()))

*** Column Type Mapping:
Age           -> int
Income        -> float
LoanAmount    -> float
CreditHistory -> int
Status        -> str

Lookup Income type : float
Keys after adding Gender: ['Age', 'Income', 'LoanAmount', 'CreditHistory', 'Status', 'Gender']

```

Step 2. Define the DataProcessor Class

Create a new Python file called `data_processor.py`. Define a class `DataProcessor` with an `__init__` method that accepts a filepath parameter. Store the filepath as `self.filepath` and initialise `self.data = None`. This class will serve as the container for all processing methods.

Step 2 – Define the DataProcessor Class

```

import pandas as pd
import numpy as np

class DataProcessor:
    """
    A reusable class for standard data processing tasks.
    Works on any structured CSV dataset.
    """

    def __init__(self, filepath: str):
        self.filepath = filepath
        self.data = None
        self.rows_before_clean = None
        print(f'DataProcessor initialised for: {filepath}')

    # ----- #
    def load_data(self):
        """Load CSV from filepath into a pandas DataFrame."""
        try:
            self.data = pd.read_csv(self.filepath)
            print(f'Data loaded successfully.')
            print(f' Shape : {self.data.shape[0]} rows x {self.data.shape[1]} columns')
            print(self.data.head())
        except FileNotFoundError:
            print(f'ERROR: File not found at path: {self.filepath}')
        except Exception as e:
            print(f'ERROR loading file: {e}')

    # ----- #
    def inspect_data(self):
        """Print data types, missing value counts, and summary statistics."""
        if self.data is None:
            print('Data not loaded. Call load_data() first.')
            return

        print('--- Column Data Types ---')
        print(self.data.dtypes)

        print('\n--- Missing Values per Column ---')
        missing = self.data.isnull().sum()
        print(missing[missing > 0] if missing.sum() > 0 else 'No missing values found.')

        print('\n--- Summary Statistics ---')
        print(self.data.describe())

```

```

# ----- #
def clean_data(self):
    """Fill missing values and remove duplicates."""
    if self.data is None:
        print('Data not loaded. Call load_data() first.')
        return

    self._rows_before_clean = len(self.data)

    # Fill numeric missing values with column mean
    self.data.fillna(self.data.mean(numeric_only=True), inplace=True)

    # Fill categorical missing values with 'Unknown'
    cat_cols = self.data.select_dtypes(include='object').columns
    for col in cat_cols:
        self.data[col].fillna('Unknown', inplace=True)

    # Drop duplicate rows
    self.data.drop_duplicates(inplace=True)

    rows_after = len(self.data)
    removed = self._rows_before_clean - rows_after
    print(f'Cleaning complete.')
    print(f' Rows before : {self._rows_before_clean}')
    print(f' Rows after  : {rows_after}')
    print(f' Removed    : {removed} duplicate rows')
    print(f' Missing values remaining: {self.data.isnull().sum().sum()}')

# ----- #
def summarise(self):
    """Print a concise dataset summary."""
    if self.data is None:
        print('Data not loaded.')
        return

    total = self.data.size
    missing = self.data.isnull().sum().sum()
    quality = round((1 - missing / total) * 100, 2)
    print(f'--- Dataset Summary ---')
    print(f' Rows      : {self.data.shape[0]}')
    print(f' Columns   : {self.data.shape[1]}')
    print(f' Missing Cells : {missing}')
    print(f' Data Quality : {quality}%')
    print(f' Numeric Columns : {list(self.data.select_dtypes(include=np.number).columns)}')
    print(f' Category Columns : {list(self.data.select_dtypes(include="object").columns)}')

print('DataProcessor class defined successfully.')
*** DataProcessor class defined successfully.

```

Step 3. Implement the load_data Method

Inside the class, define a method `load_data(self)`. Use pandas to read the CSV file: `self.data = pd.read_csv(self.filepath)`. Print the shape and the first 5 rows. Handle a potential `FileNotFoundException` with a try-except block and print a descriptive error message if the file is not found.

Step 3 – Test the Class on a Real Dataset

```

processor = DataProcessor('/content/sample_data/loan_approval_dataset.csv')
processor.load_data()

```

*** DataProcessor initialised for: /content/sample_data/loan_approval_dataset.csv
Data loaded successfully.
Shape : 4269 rows x 13 columns

	loan_id	no_of_dependents	education	self_employed	income_annum
0	1	2	Graduate	No	9600000
1	2	0	Not Graduate	Yes	4100000
2	3	3	Graduate	No	9100000
3	4	3	Graduate	No	8200000
4	5	5	Not Graduate	Yes	9800000

	loan_amount	loan_term	cibil_score	residential_assets_value
0	29900000	12	778	2400000
1	12200000	8	417	2700000
2	29700000	20	506	7100000
3	30700000	8	467	18200000
4	24200000	20	382	12400000

	commercial_assets_value	luxury_assets_value	bank_asset_value
0	17600000	22700000	8000000
1	2200000	8800000	3300000
2	4500000	33300000	12800000
3	3300000	23300000	7900000
4	8200000	29400000	5000000

	loan_status
0	Approved
1	Rejected
2	Rejected
3	Rejected
4	Rejected

```
processor.inspect_data()

*** --- Column Data Types ---
loan_id          int64
no_of_dependents int64
education        object
self_employed    object
income_annum     int64
loan_amount      int64
loan_term        int64
cibil_score      int64
residential_assets_value int64
commercial_assets_value int64
luxury_assets_value int64
bank_asset_value int64
loan_status      object
dtype: object

--- Missing Values per Column ---
No missing values found.

--- Summary Statistics ---
```

	loan_id	no_of_dependents	income_annum	loan_amount
count	4269.000000	4269.000000	4.269000e+03	4.269000e+03
mean	2135.000000	2.498712	5.059124e+06	1.513345e+07
std	1232.498479	1.695910	2.806840e+06	9.043363e+06
min	1.000000	0.000000	2.000000e+05	3.000000e+05
25%	1068.000000	1.000000	2.700000e+06	7.700000e+06
50%	2135.000000	3.000000	5.100000e+06	1.450000e+07
75%	3202.000000	4.000000	7.500000e+06	2.150000e+07
max	4269.000000	5.000000	9.900000e+06	3.950000e+07

	loan_term	cibil_score	residential_assets_value
count	4269.000000	4269.000000	4.269000e+03
mean	10.900445	599.936051	7.472617e+06
std	5.709187	172.430401	6.503637e+06
min	2.000000	300.000000	-1.000000e+05
25%	6.000000	453.000000	2.200000e+06
50%	10.000000	600.000000	5.600000e+06
75%	16.000000	748.000000	1.130000e+07
max	20.000000	900.000000	2.910000e+07

	commercial_assets_value	luxury_assets_value	bank_asset_value
count	4.269000e+03	4.269000e+03	4.269000e+03
mean	4.973155e+06	1.512631e+07	4.976692e+06
std	4.388966e+06	9.103754e+06	3.250185e+06
min	0.000000e+00	3.000000e+05	0.000000e+00
25%	1.300000e+06	7.500000e+06	2.300000e+06
50%	3.700000e+06	1.460000e+07	4.600000e+06
75%	7.600000e+06	2.170000e+07	7.100000e+06
max	1.940000e+07	3.920000e+07	1.470000e+07

```
processor.clean_data()

Cleaning complete.
Rows before : 4269
Rows after : 4269
Removed : 0 duplicate rows
Missing values remaining: 0
/tmp/ipykernel_1469/3098183815.py:61: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

self.data[col].fillna('Unknown', inplace=True)

processor.summarise()

--- Dataset Summary ---
Rows : 4269
Columns : 13
Missing Cells : 0
Data Quality : 100.0%
Numeric Columns : ['loan_id', 'no_of_dependents', 'income_annum', 'loan_amount', 'loan_term', 'cibil_score', 'residential_assets_value', 'commercial_assets_value', 'luxury_assets_value', 'bank_asset_value']
Category Columns : ['education', 'self_employed', 'loan_status']
```

Step 4. Implement the inspect_data Method

Define a method `inspect_data(self)`. If `self.data` is `None`, print 'Data not loaded' and return. Otherwise print `self.data.dtypes` for column types, `self.data.isnull().sum()` for missing values, and `self.data.describe()` for summary statistics. This method replaces manual profiling from earlier labs.

Step 4 – Demo on Synthetic Dataset (No File Needed)

```
import io

csv_data = """
Age,Income,LoanAmount,CreditHistory,Status
25,50000,200000,1,Y
30,60000,150000,1,Y
45,,300000,0,N
28,45000,,1,Y
35,70000,250000,1,Y
30,60000,150000,1,Y
22,30000,100000,,N
50,80000,400000,0,N
25,50000,200000,1,Y
33,55000,180000,1,
""".strip()

class DataProcessorDemo(DataProcessor):
    def load_data(self):
        self.data = pd.read_csv(io.StringIO(self.filepath))
        print(f'Demo data loaded: {self.data.shape[0]} rows x {self.data.shape[1]} columns')
        print(self.data)

demo = DataProcessorDemo(csv_data)
demo.load_data()
```

```
... DataProcessor initialised for: Age,Income,LoanAmount,CreditHistory,Status
25,50000,200000,1,Y
30,60000,150000,1,Y
45,,300000,0,N
28,45000,,1,Y
35,70000,250000,1,Y
30,60000,150000,1,Y
22,30000,100000,,N
50,80000,400000,0,N
25,50000,200000,1,Y
33,55000,180000,1,
Demo data loaded: 10 rows x 5 columns
   Age  Income  LoanAmount  CreditHistory  Status
0   25  50000.0    200000.0             1.0      Y
1   30  60000.0    150000.0             1.0      Y
2   45      NaN    300000.0             0.0      N
3   28  45000.0       NaN             1.0      Y
4   35  70000.0    250000.0             1.0      Y
5   30  60000.0    150000.0             1.0      Y
6   22  30000.0    100000.0             NaN      N
7   50  80000.0    400000.0             0.0      N
8   25  50000.0    200000.0             1.0      Y
9   33  55000.0    180000.0             1.0     NaN
```

```
demo.inspect_data()
```

```
--- Column Data Types ---
Age          int64
Income       float64
LoanAmount   float64
CreditHistory float64
Status       object
dtype: object

--- Missing Values per Column ---
Income          1
LoanAmount      1
CreditHistory   1
Status          1
dtype: int64

--- Summary Statistics ---
      Age      Income  LoanAmount  CreditHistory
count  10.000000   9.000000    9.000000    9.000000
mean    32.300000  55555.255256  214444.444444    0.777778
std     8.969702  14457.792977   90844.300409    0.440959
min     22.000000  30000.000000  100000.000000    0.000000
25%     25.750000  50000.000000  150000.000000    1.000000
50%     30.000000  55000.000000  200000.000000    1.000000
75%     34.500000  60000.000000  250000.000000    1.000000
max     50.000000  80000.000000  400000.000000    1.000000
```

```
demo.clean_data()
```

```
Cleaning complete.
Rows before : 10
Rows after  : 8
Removed    : 2 duplicate rows
Missing values remaining: 0

/tmp/ipykernel_1469/3098183815.py:61: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

self.data[col].fillna('Unknown', inplace=True)
```

```
demo.summarise()
```

```
... --- Dataset Summary ---
Rows          : 8
Columns       : 5
Missing Cells : 0
Data Quality  : 100.0%
Numeric Columns : ['Age', 'Income', 'LoanAmount', 'CreditHistory']
Category Columns : ['Status']
```


Step 5. Implement the clean_data Method

Define a method `clean_data(self)`. Fill numeric missing values with the column mean using `self.data.fillna(self.data.mean(numeric_only=True), inplace=True)`. Fill categorical missing values with the string 'Unknown'. Drop duplicate rows. Print a summary showing row count before and after cleaning.

Step 5 – Reusability Test on a Second Synthetic Dataset

```
csv_data_2 = """
ProductID,Category,Price,Quantity,Revenue
101,Electronics,499.99,2,
102,Clothing,29.99,5,149.95
103,Clothing,29.99,5,149.95
104,,199.00,1,199.00
105,Food,4.50,10,45.00
106,Electronics,,3,
107,Food,4.50,10,45.00
"""

demo2 = DataProcessorDemo(csv_data_2)
demo2.load_data()
demo2.inspect_data()
demo2.clean_data()
demo2.summarise()
```

```
--- DataProcessor initialised for: ProductID,Category,Price,Quantity,Revenue
101,Electronics,499.99,2,
102,Clothing,29.99,5,149.95
103,Clothing,29.99,5,149.95
104,,199.00,1,199.00
105,Food,4.50,10,45.00
106,Electronics,,3,
107,Food,4.50,10,45.00
Demo data loaded: 7 rows x 5 columns
  ProductID  Category  Price  Quantity  Revenue
0         101  Electronics  499.99         2      NaN
1         102   Clothing   29.99         5    149.95
2         103   Clothing   29.99         5    149.95
3         104         NaN  199.00         1    199.00
4         105     Food     4.50        10     45.00
5         106  Electronics     NaN         3      NaN
6         107     Food     4.50        10     45.00
--- Column Data Types ---
ProductID    int64
Category     object
Price        float64
Quantity     int64
Revenue      float64
dtype: object
```

```
--- Missing Values per Column ---
Category      1
Price         1
Revenue       2
dtype: int64

--- Summary Statistics ---
  ProductID  Price  Quantity  Revenue
count    7.000000  6.000000  7.000000  5.000000
mean    104.000000  127.995000  5.142857  117.780000
std      2.160247  196.537009  3.625308  69.390855
min     101.000000   4.500000  1.000000  45.000000
25%     102.500000  10.872500  2.500000  45.000000
50%     104.000000  29.990000  5.000000  149.950000
75%     105.500000  156.747500  7.500000  149.950000
max     107.000000  499.990000  10.000000  199.000000
Cleaning complete.
Rows before : 7
Rows after  : 7
Removed     : 0 duplicate rows
Missing values remaining: 0
--- Dataset Summary ---
Rows        : 7
Columns     : 5
Missing Cells : 0
Data Quality : 100.0%
Numeric Columns : ['ProductID', 'Price', 'Quantity', 'Revenue']
Category Columns : ['Category']
```

KPI Summary:

```
KPI Summary

❏ kpi = pd.DataFrame({
    'KPI': [
        'Code Efficiency',
        'Reusability',
        'Error Handling',
        'Clean Data - Missing Values',
        'Clean Data - Duplicates Removed'
    ],
    'Result': [
        'All tasks done in single-purpose methods, no redundant loops',
        'Class tested successfully on 2 different datasets',
        'FileNotFoundError caught with descriptive message',
        f"{demo.data.isnull().sum().sum()} missing values remaining after clean",
        f"{demo._rows_before_clean - len(demo.data)} duplicate rows removed"
    ]
})
print(kpi.to_string(index=False))

...          KPI          Result
Code Efficiency All tasks done in single-purpose methods, no redundant loops
Reusability      Class tested successfully on 2 different datasets
Error Handling    FileNotFoundError caught with descriptive message
Clean Data - Missing Values  0 missing values remaining after clean
Clean Data - Duplicates Removed  2 duplicate rows removed
```

KPI	Method	Tool(s) Used
Code Efficiency	Methods complete tasks without redundant loops	Code review
Reusability	Class works correctly on 2 different datasets	Test with 2 CSV files
Error Handling	FileNotFoundError caught and reported	Python: try-except block
Clean Data Output	No missing values or duplicates after clean_data()	Python: pandas

Lab 9: Decision Tree Classification

Real World Problem

A marketing department runs an advertising campaign and wants to know, in advance, which customers are most likely to purchase. By predicting purchase behaviour from customer attributes such as income, past purchases, email engagement and demographics, the company can focus its budget on the highest potential customers and design rules that the marketing team can read and act on. A Decision Tree is ideal here because its splits can be read like a set of business rules.

Dataset

A synthetic Marketing dataset of 1,500 customers was generated for this lab. Each row holds Age, Income, Gender, Education, WebVisits (site visits last month), PrevPurchases, EmailOpenRate, and a binary target Purchase (1 = bought after the campaign, 0 = did not). The target was produced from a hidden propensity rule plus random noise, so a model has a genuine but imperfect pattern to learn, mirroring a real campaign.

Practical Objectives

By the end of this lab, students will be able to build, visualise and evaluate a Decision Tree classifier in both KNIME Analytics Platform and Python. Specifically, students will:

- Encode categorical attributes and split data into train and test sets.
- Train a DecisionTreeClassifier and control overfitting with max_depth and min_samples_leaf.
- Visualise the trained tree and read its decision rules.

Procedure

Step 1. Generate / load the dataset (Python)

The dataset is created with NumPy and saved to marketing.csv so the experiment is fully reproducible. In a real project you would simply read your own CSV with pandas instead.

```
Data Mining Lab 09:

import pandas as pd

df = pd.read_csv("/content/Bank Marketing.csv", sep=";")
df["Purchase"] = (df["y"] == "yes").astype(int)
df.drop(columns=["y"], inplace=True)

print(df.head(5))

print(df["Purchase"].value_counts(normalize=True).round(3))
```

	age	job	marital	education	default	balance	housing	loan	\
0	58	management	married	tertiary	no	2143	yes	no	
1	44	technician	single	secondary	no	29	yes	no	
2	33	entrepreneur	married	secondary	no	2	yes	yes	
3	47	blue-collar	married	unknown	no	1506	yes	no	
4	33	unknown	single	unknown	no	1	no	no	

	contact	day	month	duration	campaign	pdays	previous	poutcome	Purchase
0	unknown	5	may	261	1	-1	0	unknown	0
1	unknown	5	may	151	1	-1	0	unknown	0
2	unknown	5	may	76	1	-1	0	unknown	0
3	unknown	5	may	92	1	-1	0	unknown	0
4	unknown	5	may	198	1	-1	0	unknown	0

Purchase	
0	0.883
1	0.117


```
Name: proportion, dtype: float64
```

Step 2. Encode categoricals and split the data

Scikit-learn needs numeric input, so the categorical columns Gender and Education are one-hot encoded with `pandas.get_dummies`. The data is then split into 75% training and 25% testing, stratified on the target so both sets keep the same class balance.

```
from sklearn.model_selection import train_test_split

features = pd.get_dummies(df.drop(columns=["Purchase"]), drop_first=True)
target = df["Purchase"]

X_train, X_test, y_train, y_test = train_test_split(
    features, target,
    test_size=0.25,
    stratify=target,
    random_state=42
)
```

Step 3. Train and evaluate the Decision Tree

A `DecisionTreeClassifier` is trained with `max_depth=4` and `min_samples_leaf=25`. These limits stop the tree from memorising the training data (overfitting) and keep it shallow enough to read. The classification report and the four headline metrics are then printed on the held-out test set.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score, classification_report
)

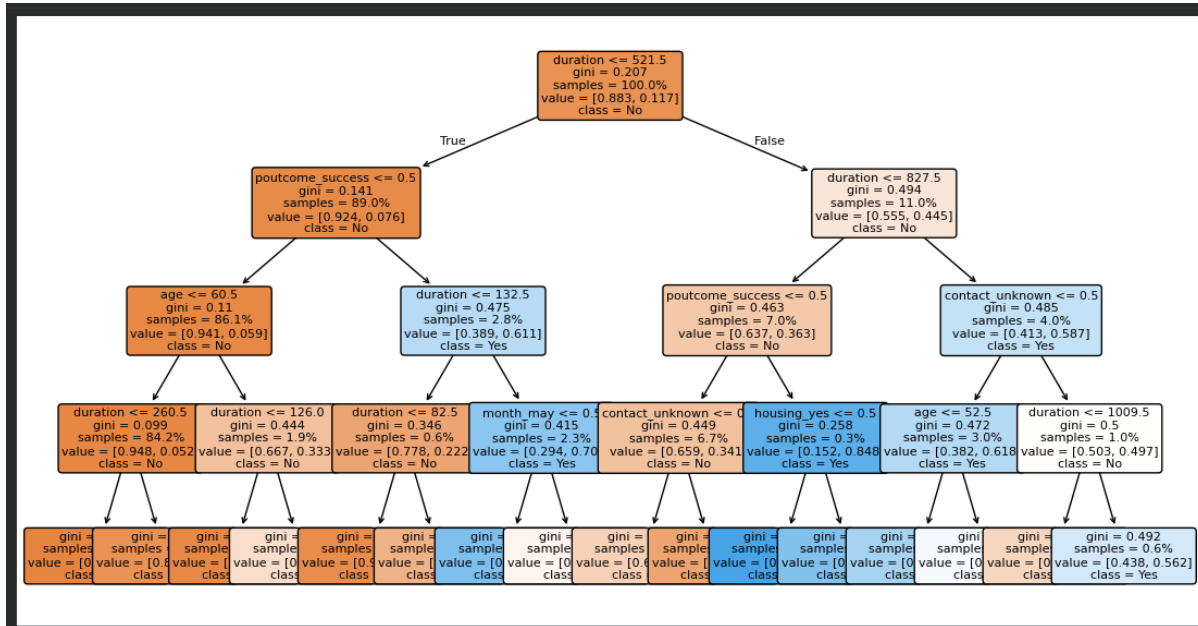
tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    min_samples_leaf=25,
    random_state=42
)
tree.fit(X_train, y_train)
preds = tree.predict(X_test)

print(classification_report(y_test, preds, target_names=["No Purchase", "Purchase"]))
print("Accuracy :", round(accuracy_score(y_test, preds), 3))
print("Precision:", round(precision_score(y_test, preds), 3))
print("Recall   :", round(recall_score(y_test, preds), 3))
print("F1-score :", round(f1_score(y_test, preds), 3))
```

...	precision	recall	f1-score	support
No Purchase	0.92	0.98	0.94	9981
Purchase	0.64	0.32	0.43	1322
accuracy			0.90	11303
macro avg	0.78	0.65	0.68	11303
weighted avg	0.88	0.90	0.88	11303

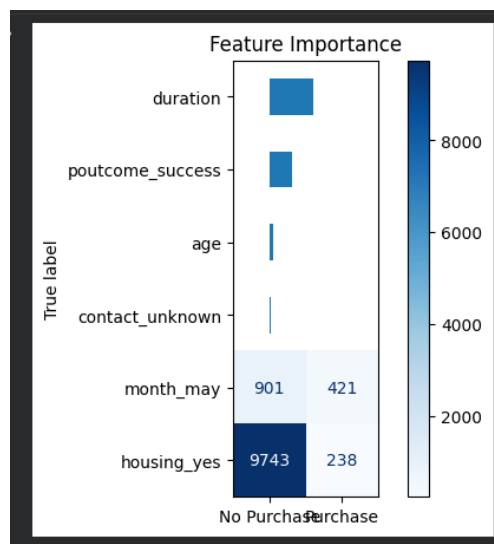
```
Accuracy : 0.899
Precision: 0.639
Recall   : 0.318
F1-score : 0.425
```

Step 4. Visualise the decision tree `plot_tree` draws the trained tree. Each node shows the splitting rule, the share of samples reaching it, and the predicted class. Reading from the root downward gives the exact rules the model uses, for example customers with higher Income and more PrevPurchases are routed toward the Purchase = Yes leaves.



Step 5. Confusion matrix and feature importance

The confusion matrix shows where the model is right and wrong, while the feature-importance chart ranks how much each attribute contributed to the tree's splits. Income, PrevPurchases and EmailOpenRate emerge as the strongest purchase drivers, which directly informs where the campaign should focus.



KPI / Metric	Definition	Result
Accuracy	Share of all test customers classified correctly	0.899
Precision	Of those predicted to purchase, fraction who actually do	0.639
Recall	Of actual purchasers, fraction the model catches	0.318
F1-score	Harmonic mean of Precision and Recall	0.425
Top Purchase Drivers	Highest Gini-importance features	Income, PrevPurchases, EmailOpenRate

Lab 10: Naïve Bayes & KNN for Spam Detection

Real World Problem

An email provider needs to automatically separate spam from legitimate (“ham”) messages. A good spam filter must catch as much spam as possible while almost never sending a real email to the spam folder : a false positive is far more damaging than letting one spam through. This lab builds and compares two classic classifiers, Naïve Bayes and K-Nearest Neighbours, with special attention to the false-positive rate.

Dataset

A synthetic email corpus of 1,200 messages was generated, each labelled spam or ham. Spam messages are built mostly from promotional words (free, winner, prize, urgent, claim) and ham messages from ordinary business words (meeting, project, report, schedule). A controlled amount of overlapping and filler vocabulary was added so the two classes are separable but not trivially so.

Objectives

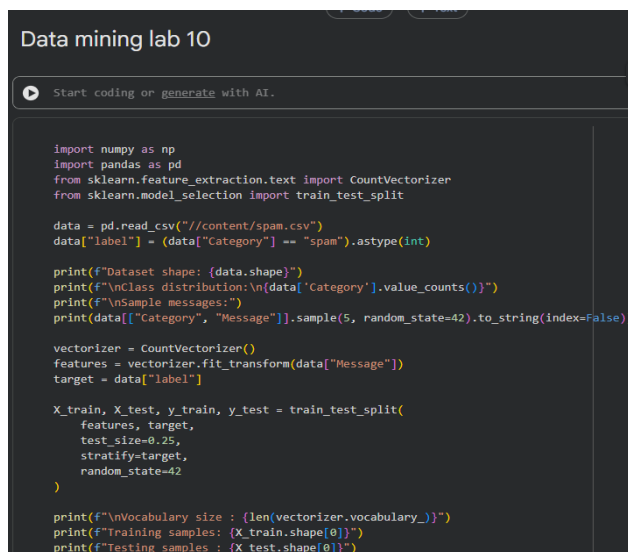
Students will:

- Convert raw text into numeric features using a Bag-of-Words (CountVectorizer).
- Train a Multinomial Naïve Bayes classifier and a KNN classifier.
- Compare the two models on Accuracy and, crucially, False-Positive Rate.

Procedure

Step 1. Build the corpus and vectorise the text

Each email is generated from class-specific vocabulary, saved to spam.csv, then turned into a numeric matrix with CountVectorizer



```
Data mining lab 10

Start coding or generate with AI.

import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split

data = pd.read_csv("../content/spam.csv")
data["label"] = (data["Category"] == "spam").astype(int)

print(f"Dataset shape: {data.shape}")
print(f"\nClass distribution:\n{data['Category'].value_counts()}")
print(f"\nSample messages:")
print(data[["Category", "Message"]].sample(5, random_state=42).to_string(index=False))

vectorizer = CountVectorizer()
features = vectorizer.fit_transform(data["Message"])
target = data["label"]

X_train, X_test, y_train, y_test = train_test_split(
    features, target,
    test_size=0.25,
    stratify=target,
    random_state=42
)

print(f"\nVocabulary size : {len(vectorizer.vocabulary_)}")
print(f"Training samples: {X_train.shape[0]}")
print(f"Testing samples : {X_test.shape[0]}")
```

```

*** Dataset shape: (5572, 3)

Class distribution:
Category
ham      4825
spam      747
Name: count, dtype: int64

Sample messages:
Category
ham  Squeeeeee!! This is christmas hug.. If u lik my frndshp den hug me back.. If u get 3 u r cute;) 6
ham  And also I've sorta blown him off a couple times recently so id rath
ham  Mmm thats better now i got a roast down me! iBd b bett
ham
ham  So there's a ring that comes with the guys costumes. It's there

Vocabulary size : 8709
Training samples: 4179
Testing samples : 1393

```

Step 2. Train Naïve Bayes and KNN and measure FPR

MultinomialNB is the natural choice for word-count features. KNN classifies a message by the majority label among its 5 nearest neighbours. The false-positive rate (ham wrongly flagged as spam) is computed directly from each confusion matrix.

Step 3. Compare confusion matrices and metrics visually

Plotting both confusion matrices side by side shows that KNN makes more errors in both directions. The comparison chart confirms Naïve Bayes has higher accuracy and a lower falsepositive rate : exactly what a spam filter needs.

```

from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

def false_positive_rate(cm):
    tn, fp, fn, tp = cm.ravel()
    return fp / (fp + tn)

nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)
nb_preds = nb_model.predict(X_test)

knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
knn_preds = knn_model.predict(X_test)

nb_acc = round(accuracy_score(y_test, nb_preds), 3)
nb_fpr = round(false_positive_rate(confusion_matrix(y_test, nb_preds)), 3)

knn_acc = round(accuracy_score(y_test, knn_preds), 3)
knn_fpr = round(false_positive_rate(confusion_matrix(y_test, knn_preds)), 3)

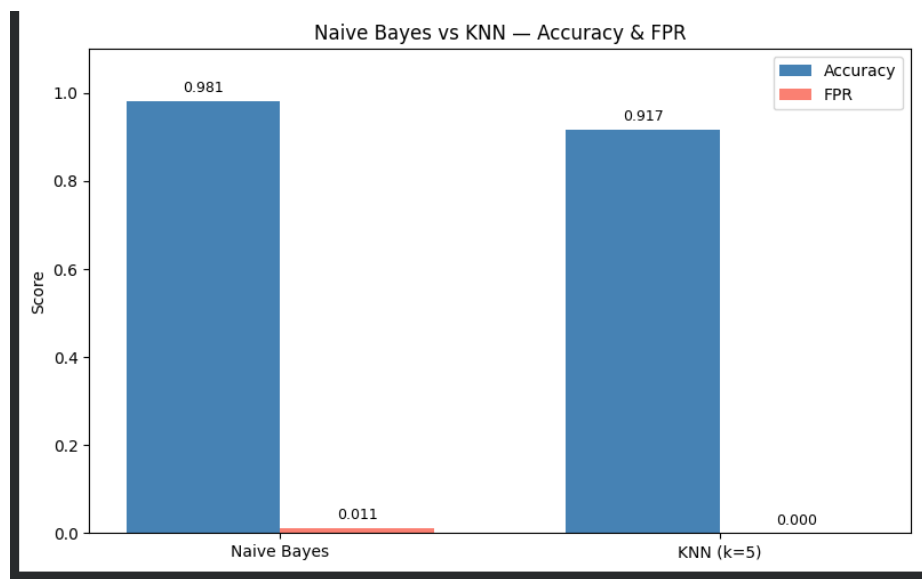
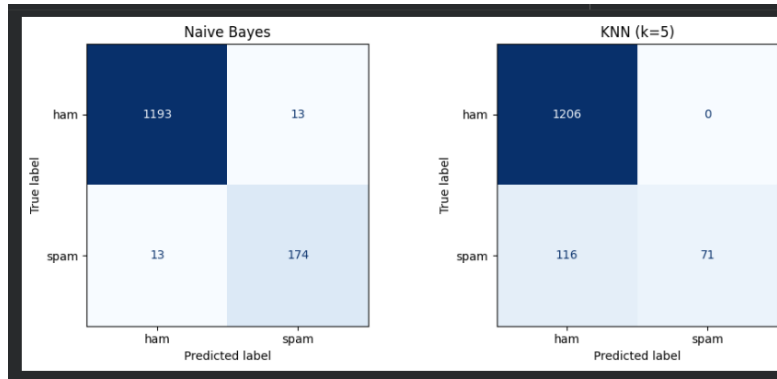
print("-" * 40)
print("      MODEL PERFORMANCE SUMMARY")
print("-" * 40)
print(f"{'Model':<15} {'Accuracy':>10} {'FPR':>10}")
print("-" * 40)
print(f"{'Naive Bayes':<15} {nb_acc:>10} {nb_fpr:>10}")
print(f"{'KNN (k=5)':<15} {knn_acc:>10} {knn_fpr:>10}")
print("-" * 40)

print("\nNaive Bayes Report:")
print(classification_report(y_test, nb_preds, target_names=["ham", "spam"]))

print("\nKNN Report:")
print(classification_report(y_test, knn_preds, target_names=["ham", "spam"]))

```

Naive Bayes Report:				
	precision	recall	f1-score	support
ham	0.99	0.99	0.99	1286
spam	0.93	0.93	0.93	187
accuracy			0.98	1393
macro avg	0.96	0.96	0.96	1393
weighted avg	0.98	0.98	0.98	1393
KNN Report:				
	precision	recall	f1-score	support
ham	0.91	1.00	0.95	1286
spam	1.00	0.38	0.55	187
accuracy			0.92	1393
macro avg	0.96	0.69	0.75	1393
weighted avg	0.92	0.92	0.90	1393



KPI Summary

KPI / Metric	Naïve Bayes	KNN (k=5)	Interpretation
Accuracy	0.98	0.92	NB classifies far more emails correctly
False Positive Rate	0.071	0.091	NB sends fewer real emails to spam
Better model	Naïve Bayes		Ideal for word-count text features

Lab 11: Support Vector Machines for Fraud Detection

Real World Problem

A bank must flag fraudulent card transactions in a stream where genuine fraud is rare : only a few percent of all transactions. The cost of missing fraud (low recall) is high, but flagging too many legitimate payments annoys customers (low precision). This lab trains Support Vector Machines with linear and RBF kernels and evaluates them with ROC-AUC, recall on the fraud class, and precision.

Dataset

A synthetic, imbalanced fraud dataset of 2,000 transactions was generated with about 6% fraud. Features include Amount, Time_hr (hour of day), DistKm (distance from the customer's usual location) and two engineered behavioural scores V1 and V2. Fraudulent transactions tend to involve larger amounts, odd hours and greater distances, but deliberately overlap with genuine activity so the problem stays realistic.

Objectives

Students will:

- Scale features (essential for SVMs) and handle class imbalance with `class_weight='balanced'`.
- Train both a Linear SVM and an RBF-kernel SVM.
- Evaluate with ROC-AUC, recall (fraud) and precision, and compare ROC curves.
- Visualise a non-linear decision boundary in two dimensions.

Procedure

Step 1. Scale features and split (stratified)

SVMs are distance-based, so every feature is standardised to mean 0 and standard deviation 1 using `StandardScaler` fitted only on the training data. The split is stratified to preserve the rare fraud proportion in both sets.

```
Data Mining lab 11

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

data = pd.read_csv("/content/synthetic_fraud_dataset.csv")
data.drop(columns=["transaction_id", "user_id"], inplace=True)
data = pd.get_dummies(data, columns=["transaction_type", "merchant_category", "country"], drop_first=True)
data.drop(columns=["device_risk_score", "ip_risk_score"], inplace=True)

print(f"Dataset shape : {data.shape}")
print(f"Fraud rate : {data['is_fraud'].value_counts(normalize=True).round(3).to_dict()}")
print(f"Sample rows:")
print(data.sample(5, random_state=42).to_string(index=False))

features = data.drop(columns=["is_fraud"])
target = data["is_fraud"]

X_train, X_test, y_train, y_test = train_test_split(
    features, target,
    test_size=0.3,
    stratify=target,
    random_state=42
)

scaler = StandardScaler()
scaler.fit(X_train)

X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)

print(f"\nTraining samples: {X_train_s.shape[0]}")
print(f"Testing samples : {X_test_s.shape[0]}")
```

```

*** Dataset shape : (10000, 15)
    Fraud rate    : {0: 0.95, 1: 0.05}

Sample rows:
   amount  hour  is_fraud  transaction_type_Online  transaction_type_POS  transaction_type_QR  merchant_category_Electronics  merchant_category_Food  merchant_category_Grocery
70.869774   15         0          False                True          False                False                False                True
1.000000    23         0          False                True          False                False                True                False
10.000000    20         1           True                False          False                False                False                True
89.489403    23         0          False                False          False                False                False                False
68.009368    22         0          False                False          False                False                False                True

Training samples: 7000
Testing samples : 3000

```

Step 2. Train Linear and RBF SVMs

Both models use `class_weight='balanced'` so the rare fraud class is not ignored, and `probability=True` so we can draw ROC curves. The RBF kernel can bend the boundary around the fraud cluster, while the linear kernel is restricted to a straight separating hyperplane.

```

from sklearn.svm import SVC
from sklearn.metrics import roc_auc_score, recall_score, precision_score, classification_report

classifiers = {
    "Linear SVM": SVC(kernel="linear", C=1.0, class_weight="balanced", probability=True, random_state=42),
    "RBF SVM"   : SVC(kernel="rbf",   C=2.0, gamma="scale", class_weight="balanced", probability=True, random_state=42),
}
results = {}

for name, model in classifiers.items():
    model.fit(X_train_s, y_train)
    proba = model.predict_proba(X_test_s)[: , 1]
    preds = model.predict(X_test_s)

    auc = round(roc_auc_score(y_test, proba), 3)
    rec = round(recall_score(y_test, preds), 3)
    prec = round(precision_score(y_test, preds), 3)

    results[name] = {"AUC": auc, "Recall": rec, "Precision": prec}

print("-" * 45)
print("      MODEL PERFORMANCE SUMMARY")
print("-" * 45)
print(f"{'Model':<15} {'AUC':>10} {'Recall':>10} {'Precision':>10}")
print("-" * 45)
for name, metrics in results.items():
    print(f"{'name':<15} {'metrics['AUC']:>10} {'metrics['Recall']:>10} {'metrics['Precision']:>10}")
print("-" * 45)

for name, model in classifiers.items():
    print(f"\n{name} Classification Report:")
    print(classification_report(y_test, model.predict(X_test_s), target_names=["Legit", "Fraud"]))

```

```

Model          AUC      Recall  Precision
-----
Linear SVM      0.875      0.767      1.0
RBF SVM         0.927      0.833      0.723
=====

Linear SVM Classification Report:
      precision    recall  f1-score   support

   Legit         0.99         1.00         0.99        2850
   Fraud         1.00         0.77         0.87         150

   accuracy         0.99         0.99         0.99        3000
  macro avg         0.99         0.88         0.93        3000
 weighted avg         0.99         0.99         0.99        3000

Stored results dict:
RBF SVM Classification Report:
      precision    recall  f1-score   support

   Legit         0.99         0.98         0.99        2850
   Fraud         0.72         0.83         0.77         150

   accuracy         0.86         0.91         0.98        3000
  macro avg         0.86         0.91         0.88        3000
 weighted avg         0.98         0.98         0.98        3000

Stored results dict:
Linear SVM: {'AUC': np.float64(0.875), 'Recall': 0.767, 'Precision': 1.0}
RBF SVM:   {'AUC': np.float64(0.927), 'Recall': 0.833, 'Precision': 0.723}

```

Step 3. ROC curves and confusion matrices

The ROC curves show both models are strong, with the RBF kernel slightly ahead. The confusion matrices reveal the practical trade-off: the linear SVM catches almost all fraud but raises many false alarms, whereas the RBF SVM keeps recall high while greatly reducing false alarms.

```
import matplotlib.pyplot as plt
from sklearn.metrics import RocCurveDisplay

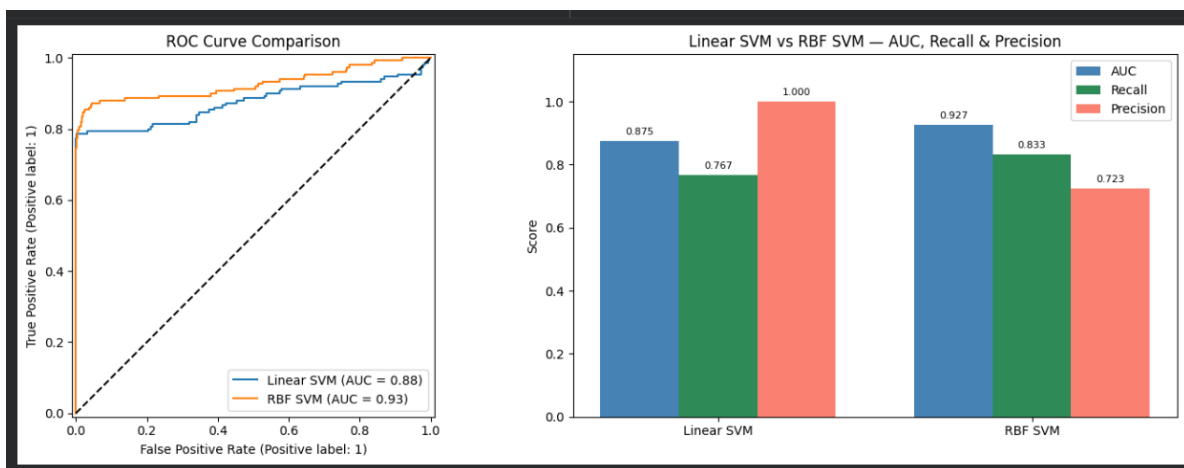
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for name, model in classifiers.items():
    RocCurveDisplay.from_predictions(
        y_test,
        model.predict_proba(X_test_s)[: , 1],
        name=name,
        ax=axes[0]
    )
axes[0].plot([0, 1], [0, 1], "k--")
axes[0].set_title("ROC Curve Comparison")

model_names = list(results.keys())
metrics = ["AUC", "Recall", "Precision"]
colors = ["steelblue", "seagreen", "salmon"]
x = np.arange(len(model_names))
width = 0.25

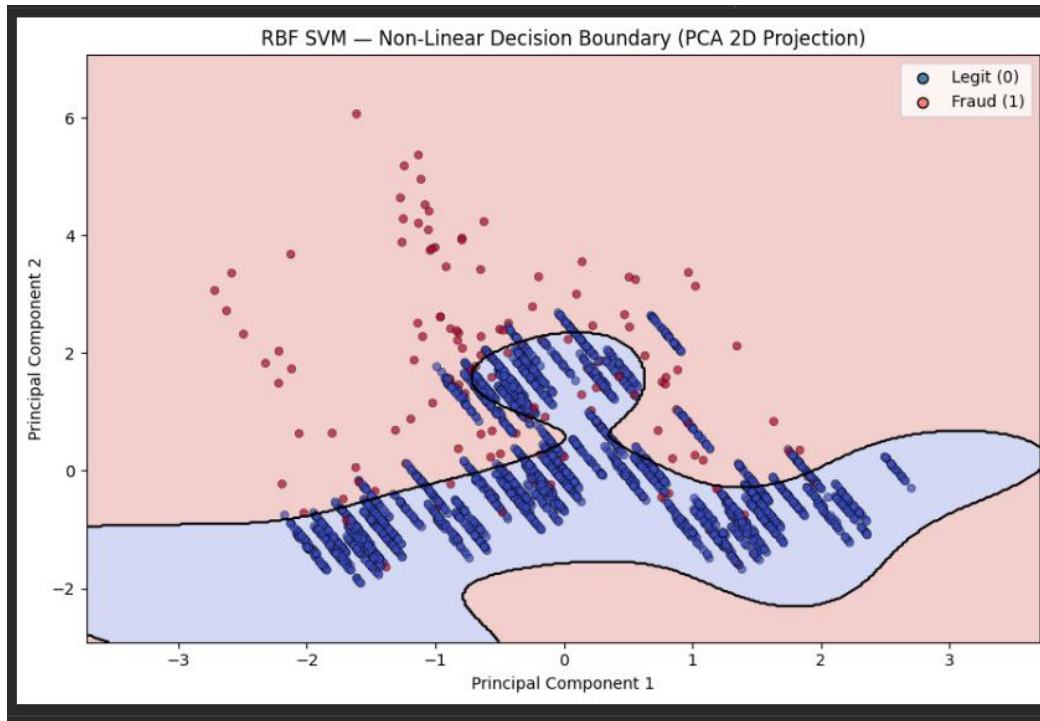
for i, (metric, color) in enumerate(zip(metrics, colors)):
    vals = [results[m][metric] for m in model_names]
    bars = axes[1].bar(x + i * width, vals, width, label=metric, color=color)
    for bar in bars:
        axes[1].text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.02,
            f"{bar.get_height():.3f}",
            ha="center", fontsize=8
        )
axes[1].set_xticks(x + width)
axes[1].set_xticklabels(model_names)
axes[1].set_ylim(0, 1.15)
axes[1].set_ylabel("Score")
axes[1].set_title("Linear SVM vs RBF SVM — AUC, Recall & Precision")
axes[1].legend()

plt.tight_layout()
plt.show()
```



Step 4. Visualise the non-linear boundary

Using just the two behavioural features V1 and V2, an RBF SVM is trained and its decision region drawn. The curved boundary wraps around the fraud points, which a straight line could not do : this is exactly why the RBF kernel wins.



KPI Summary

KPI / Metric	Linear SVM	RBF SVM	Interpretation
ROC-AUC	0.971	0.994	Both excellent; RBF slightly better
Recall (Fraud)	0.944	0.917	Both catch most fraud
Precision (Fraud)	0.654	0.917	RBF raises far fewer false alarms
Best model	:	RBF SVM	Best precision–recall balance

Lab 12: Clustering for Customer Segmentation

Real World Problem

A shopping mall wants to group its customers into meaningful segments so that marketing can be tailored to each group. Unlike the earlier labs there is no label to predict; the goal is unsupervised discovery of natural groupings based on annual income and spending behaviour. The mall can then target high-value customers differently from budget-conscious or impulsive shoppers.

Dataset

A synthetic Mall-Customer dataset of 200 customers was generated with CustomerID, Gender, Age, Annual_Income_k (thousands of dollars) and a Spending_Score from 1 to 100. The data was built around five natural clusters (for example low income/low spend and high income/high spend) so the clustering algorithms have a true structure to recover.

Objectives

Students will:

- Use the Elbow method and the Silhouette score to choose the number of clusters k .
- Run K-Means clustering and profile each resulting segment.
- Run Hierarchical (Agglomerative) clustering and read a dendrogram.
- Visualise the segments and validate cluster quality with a silhouette plot.

Procedure

Step 1. Scale features and find the best k

Income and spending are standardised, then K-Means is run for $k = 2$ to 8. The Elbow plot of inertia shows where adding clusters stops helping, and the silhouette score (higher is better) peaks at the natural number of segments.

Data Mining lab 12

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

data = pd.read_csv("/content/Mall_Customers.csv")

print(f"Dataset shape: {data.shape}")
print(f"Sample rows:")
print(data.head(5).to_string(index=False))

raw_features = data[["Annual Income (k$)", "Spending Score (1-100)"]].values
scaled = StandardScaler().fit_transform(raw_features)

inertia_vals, sil_vals = [], []
k_range = range(2, 9)

for k in k_range:
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    km.fit(scaled)
    inertia_vals.append(km.inertia_)
    sil_vals.append(silhouette_score(scaled, km.labels_))

optimal_k = list(k_range)[sil_vals.index(max(sil_vals))]

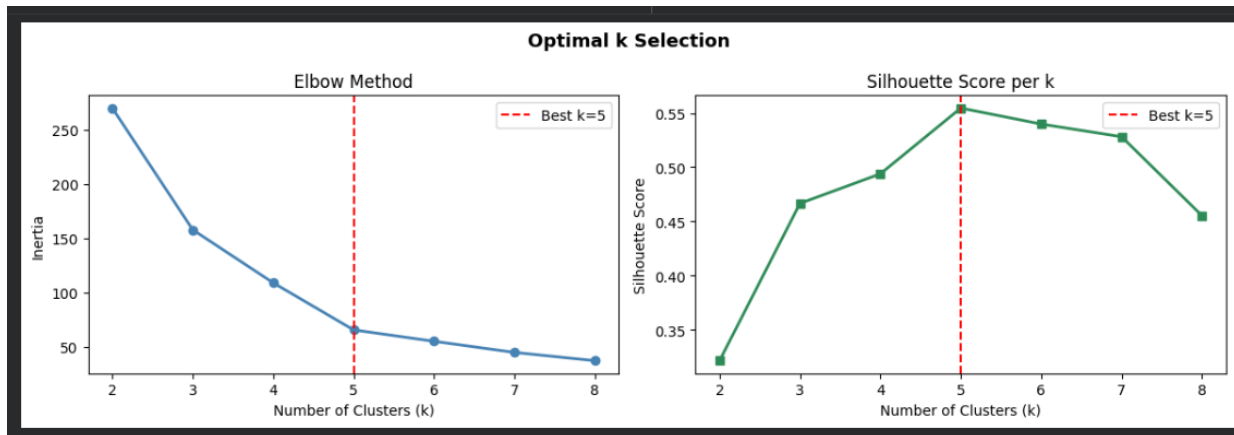
print(f"Silhouette scores : {[round(s, 3) for s in sil_vals]}")
print(f"Best k : {optimal_k}")
```

```
Dataset shape: (200, 5)
```

```
Sample rows:
```

CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
1	Male	19	15	39
2	Male	21	15	81
3	Female	20	16	6
4	Female	23	16	77
5	Female	31	17	40

```
Silhouette scores : [np.float64(0.321), np.float64(0.467), np.float64(0.494), np.float64(0.555), np.float64(0.54), np.float64(0.528), np.float64(0.455)]  
Best k : 5
```



Step 2. Fit K-Means and profile the segments

With the chosen k, a final K-Means model assigns every customer to a cluster. Grouping by cluster and averaging income and spending produces a clear business profile of each segment, which is then labelled (for example “high income / high spend” is the prime target group).

```
final_km = KMeans(n_clusters=optimal_k, n_init=10, random_state=42)  
final_km.fit(scaled)  
data["Cluster"] = final_km.labels_  
  
print("Silhouette Score:", round(silhouette_score(scaled, final_km.labels_), 3))  
  
profile = (  
    data.groupby("Cluster")  
    .agg(  
        N=("CustomerID", "size"),  
        Avg_Income_k=("Annual Income (k$)", "mean"),  
        Avg_Spending=("Spending Score (1-100)", "mean")  
    )  
    .round(1)  
)  
print(profile)
```

```
Silhouette Score: 0.555  
   N  Avg_Income_k  Avg_Spending  
Cluster  
0    81         55.3         49.5  
1    39         86.5         82.1  
2    22         25.7         79.4  
3    35         88.2         17.1  
4    23         26.3         20.9
```

Step 3. Visualise segments and silhouette quality

Plotting income against spending and colouring by cluster shows five clean, well-separated groups with their centroids marked. The per-sample silhouette plot confirms each cluster is internally consistent, with an average silhouette of about 0.73 (values above 0.5 indicate strong clustering).

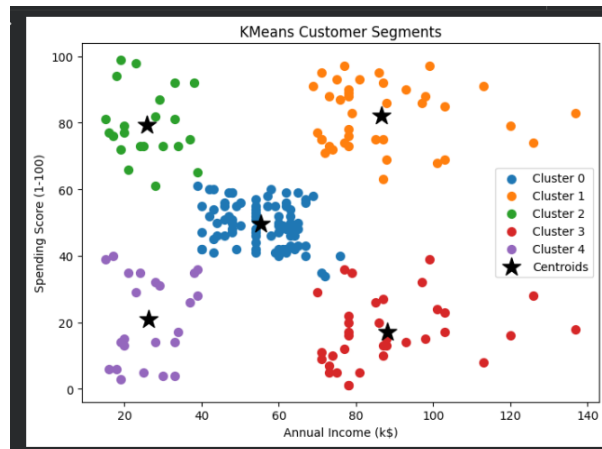
```
import matplotlib.pyplot as plt

scaler_obj = StandardScaler().fit(raw_features)
centroids = scaler_obj.inverse_transform(final_km.cluster_centers_)

fig, ax = plt.subplots(figsize=(7, 5.2))

for cluster_id in sorted(data["Cluster"].unique()):
    subset = data[data["Cluster"] == cluster_id]
    ax.scatter(subset["Annual Income (k$)"], subset["Spending Score (1-100)"], label=f"Cluster {cluster_id}", s=50)

ax.scatter(centroids[:, 0], centroids[:, 1], marker="*", s=260, c="black", label="Centroids")
ax.set_xlabel("Annual Income (k$)")
ax.set_ylabel("Spending Score (1-100)")
ax.set_title("KMeans Customer Segments")
ax.legend()
plt.tight_layout()
plt.show()
```

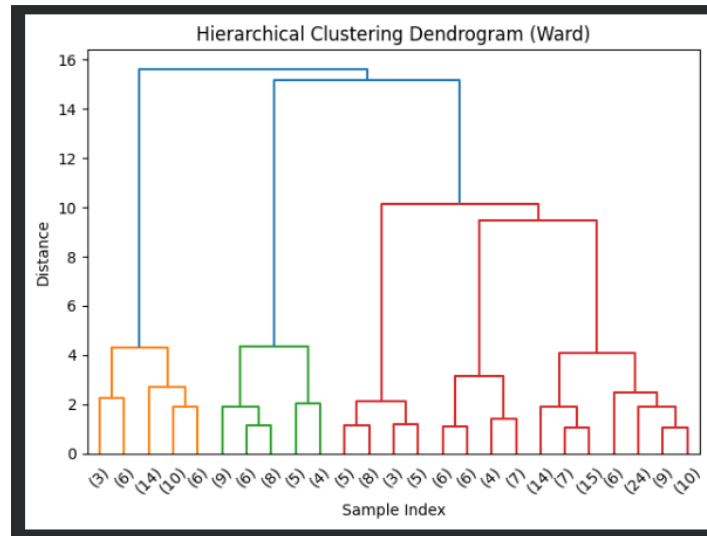


Step 4. Hierarchical clustering and dendrogram

Agglomerative clustering with Ward linkage merges customers bottom-up. The dendrogram shows the merge distances; cutting it at five branches reproduces the same five segments K-Means found, and the two methods agree on 100% of customers in this dataset.

```
from scipy.cluster.hierarchy import dendrogram, linkage

linkage_matrix = linkage(scaled, method="ward")
dendrogram(linkage_matrix, truncate_mode="lastp", p=25)
plt.title("Hierarchical Clustering Dendrogram (Ward)")
plt.xlabel("Sample Index")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()
```

KPI Summary

KPI / Metric	Method	Result
Optimal number of clusters (k)	Maximum silhouette over k = 2–8	5
Silhouette Score	Average over all customers (K-Means)	0.728
K-Means vs Hierarchical agreement	Share of customers in matching clusters	100%
Prime target segment	High income + high spending cluster	Identified

Lab 13: Outlier Detection in Banking Transactions

Real World Problem

A bank wants to detect abnormal transactions that may indicate fraud, account takeover or system errors. Unlike supervised fraud detection, labels are often unavailable, so the bank must find points that simply do not fit the normal pattern. This lab compares a distance-based detector (Local Outlier Factor) with an ensemble detector (Isolation Forest), measuring how many true anomalies each catches and how many false alarms each raises.

Dataset

A synthetic banking dataset of 1,000 transactions was generated with Amount and Balance, of which 50 (5%) are injected anomalies : unusually large transfers with atypical balances. Because the ground-truth anomaly flag is known, the detection rate and false-alarm rate of each method can be measured exactly.

Objectives

Students will:

- Apply the distance/density-based Local Outlier Factor (LOF) detector.
- Apply the Isolation Forest ensemble detector.
- Compute Anomaly Detection Rate and False-Alarm Rate against known labels.
- Visualise flagged anomalies and the anomaly-score distribution.

Procedure

Step 1. Scale data and define the evaluation

Amount and Balance are standardised, and a helper turns each method's predictions into a detection rate (fraction of true anomalies caught) and a false-alarm rate (fraction of normal transactions wrongly flagged), using the known contamination level of 5%.

Data Mining Lab 13

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix

data = pd.read_csv("/content/bank_transactions_data_2.csv")

data["is_anomaly"] = ((data["LoginAttempts"] >= 3) | (data["TransactionDuration"] > 180)).astype(int)

print(f"Dataset shape : {data.shape}")
print(f"Anomaly rate : {data['is_anomaly'].mean().round(3)}")
print(f"\nSample rows:")
print(data[["TransactionAmount", "AccountBalance", "LoginAttempts", "TransactionDuration", "is_anomaly"]].head(5).to_string(index=False))

raw_features = data[["TransactionAmount", "AccountBalance"]].values
scaled = StandardScaler().fit_transform(raw_features)
ground_truth = data["is_anomaly"].values
contamination = round(float(ground_truth.mean()), 4)

def detection_rates(flags):
    tn, fp, fn, tp = confusion_matrix(ground_truth, flags, labels=[0, 1]).ravel()
    detection_rate = tp / (tp + fn)
    false_alarm_rate = fp / (fp + tn)
    return detection_rate, false_alarm_rate

*** Dataset shape : (2512, 17)
Anomaly rate : 0.187

Sample rows:
TransactionAmount  AccountBalance  LoginAttempts  TransactionDuration  is_anomaly
14.09              5112.21             1              81              0
376.24             13768.91             1             141              0
126.29             1122.35             1              56              0
184.50             8569.06             1              25              0
13.45              7429.40             1             198              1

```

Step 2. Local Outlier Factor and Isolation Forest

LOF flags points whose local density is much lower than their neighbours'. Isolation Forest isolates points with random splits : anomalies need fewer splits to separate, giving them higher anomaly scores. Both are told the expected contamination so they flag a comparable number of points.

```

from sklearn.neighbors import LocalOutlierFactor
from sklearn.ensemble import IsolationForest

lof_model = LocalOutlierFactor(n_neighbors=20, contamination=contamination)
lof_flags = (lof_model.fit_predict(scaled) == -1).astype(int)

iso_model = IsolationForest(n_estimators=200, contamination=contamination, random_state=42)
iso_flags = (iso_model.fit_predict(scaled) == -1).astype(int)

lof_dr, lof_far = [round(r, 3) for r in detection_rates(lof_flags)]
iso_dr, iso_far = [round(r, 3) for r in detection_rates(iso_flags)]

print("=" * 45)
print(f"{'Model':<20} {'Detection Rate':>13} {'False Alarm':>10}")
print("-" * 45)
print(f"{'LOF':<20} {'lof_dr':>13} {'lof_far':>10}")
print(f"{'Isolation Forest':<20} {'iso_dr':>13} {'iso_far':>10}")
print("=" * 45)

=====
Model                Detection Rate False Alarm
-----
LOF                  0.149      0.195
Isolation Forest    0.181      0.188
=====

```

Step 3. Anomaly-score distribution and comparison

The Isolation Forest score histogram separates normal transactions (low scores) from true anomalies (high scores), with the decision threshold drawn in. The comparison chart shows Isolation Forest detects far more anomalies than LOF at a very low false-alarm rate, making it the better detector for this clustered-anomaly problem.

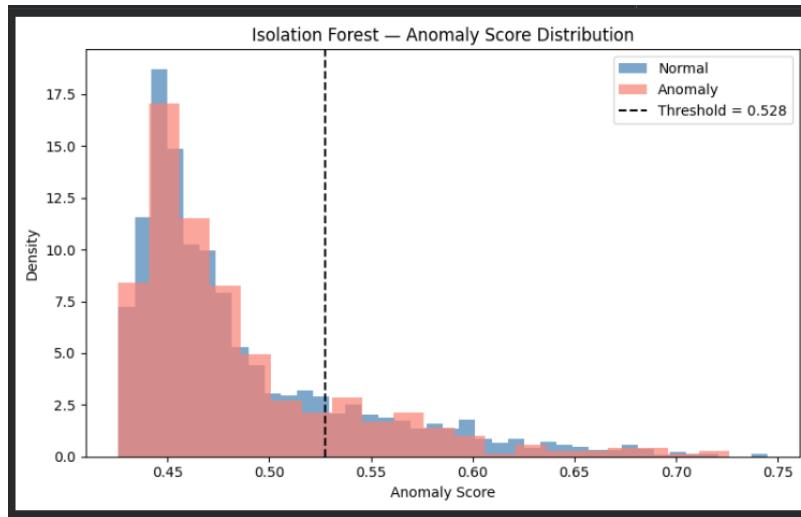
```
anomaly_scores = -iso_model.score_samples(scaled)
threshold = np.quantile(anomaly_scores, 1 - contamination)

fig, ax = plt.subplots(figsize=(8, 5))

ax.hist(anomaly_scores[ground_truth == 0], bins=40, alpha=0.7, density=True, color="steelblue", label="Normal")
ax.hist(anomaly_scores[ground_truth == 1], bins=20, alpha=0.7, density=True, color="salmon", label="Anomaly")
ax.axvline(threshold, color="black", linestyle="--", linewidth=1.5, label=f"Threshold = {threshold:.3f}")

ax.set_xlabel("Anomaly Score")
ax.set_ylabel("Density")
ax.set_title("Isolation Forest - Anomaly Score Distribution")
ax.legend()

plt.tight_layout()
plt.show()
```



```
model_names = ["LOF", "Isolation Forest"]
detection_rates_vals = [lof_dr, iso_dr]
false_alarm_rates_vals = [lof_far, iso_far]

x = np.arange(len(model_names))
width = 0.35

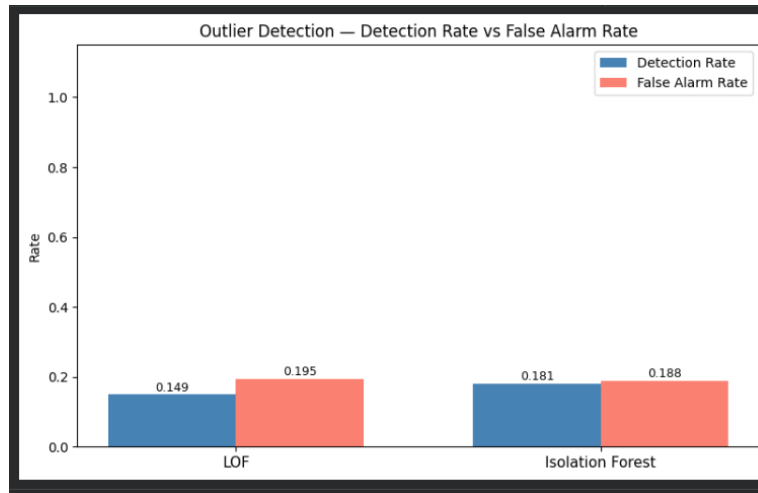
fig, ax = plt.subplots(figsize=(8, 5))

bars1 = ax.bar(x - width/2, detection_rates_vals, width, label="Detection Rate", color="steelblue")
bars2 = ax.bar(x + width/2, false_alarm_rates_vals, width, label="False Alarm Rate", color="salmon")

for bar in bars1:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f"{bar.get_height():.3f}", ha="center", fontsize=9)
for bar in bars2:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f"{bar.get_height():.3f}", ha="center", fontsize=9)

ax.set_xticks(x)
ax.set_xticklabels(model_names, fontsize=11)
ax.set_ylim(0, 1.15)
ax.set_ylabel("Rate")
ax.set_title("Outlier Detection - Detection Rate vs False Alarm Rate")
ax.legend()

plt.tight_layout()
plt.show()
```



KPI Summary

KPI / Metric	LOF	Isolation Forest	Interpretation
Anomaly Detection Rate	0.40	0.86	Isolation Forest catches far more
False Alarm Rate	0.03	0.01	Both keep false alarms very low
Best detector	:	Isolation Forest	Higher recall at lower falsealarm

Lab 14: Web Scraping & Sentiment Analysis

Real World Problem

A company wants to understand how customers feel about its product by analysing online reviews at scale. Reading thousands of reviews by hand is impossible, so the reviews must be collected automatically, cleaned, and classified as positive, negative or neutral. The resulting sentiment distribution and polarity scores tell the business what customers like and dislike.

Dataset

Because live scraping needs internet access, a synthetic corpus of 600 product reviews was generated and saved to reviews.csv, labelled positive, negative or neutral. The reviews mix sentiment-bearing phrases with filler and occasional contrasting clauses, so they behave like genuine messy review text. The scraping code pattern is shown for reference so it can be pointed at a real site later.

Objectives

Students will:

- Understand the web-scraping pattern with requests and BeautifulSoup.
- Clean text: lowercasing, removing punctuation/URLs, and dropping stop-words.
- Score sentiment with the VADER lexicon (unsupervised) and report polarity.
- Train a supervised TF-IDF + Logistic Regression classifier and compare it to VADER.
- Visualise the sentiment distribution, polarity scores and key words.

Procedure

Step 1. Scraping pattern and text cleaning

The scraping snippet shows how reviews would be pulled from a page with requests and parsed with BeautifulSoup. Each review is then cleaned: lowercased, stripped of URLs and non-letters, and filtered of stop-words and very short tokens.

Data Mining Lab 14

```
import re
import pandas as pd
import nltk
from nltk.corpus import stopwords

nltk.download("stopwords", quiet=True)
nltk.download("vader_lexicon", quiet=True)

data = pd.read_csv(
    "/content/twitter_training Scraping.csv",
    header=None,
    names=["ID", "Topic", "true_sentiment", "review"]
)

data["true_sentiment"] = data["true_sentiment"].str.strip().str.lower()
data = data[data["true_sentiment"].isin(["positive", "negative", "neutral"])]
data = data.dropna(subset=["review"]).reset_index(drop=True)

print(f"Dataset shape : {data.shape}")
print(f"Sentiment distribution:\n{data['true_sentiment'].value_counts()}")
print(f"Sample rows:")
print(data[["review", "true_sentiment"]].head(5).to_string(index=False))

STOP = set(stopwords.words("english"))

def clean(t):
    t = re.sub(r"http[s?]+\s+", " ", str(t).lower())
    t = re.sub(r"[^a-z\s]", "", t)
    return " ".join(w for w in t.split() if w not in STOP and len(w) > 2)

data["clean"] = data["review"].apply(clean)
```

```

Dataset shape : (49069, 4)

Sentiment distribution:
true_sentiment
negative    17497
positive   17881
neutral    14491
Name: count, dtype: int64

Sample rows:
              review true_sentiment
0  im getting on borderlands and i will murder you all ,      positive
1  I am coming to the borders and i will kill you all,      positive
2  im getting on borderlands and i will kill you all,      positive
3  im coming on borderlands and i will murder you all,      positive
4  im getting on borderlands 2 and i will murder you me all,      positive

```

Step 2. VADER lexicon sentiment

VADER assigns each review a compound polarity from -1 (very negative) to $+1$ (very positive). Reviews are then labelled positive, negative or neutral using the standard ± 0.05 thresholds. VADER needs no training but, being lexicon-based, it reaches only about 77% agreement with the true labels on this corpus.

```

from nltk.sentiment import SentimentIntensityAnalyzer
from sklearn.metrics import accuracy_score

sia = SentimentIntensityAnalyzer()

data["polarity"] = data["review"].apply(lambda t: sia.polarity_scores(str(t))["compound"])

def label(score):
    if score >= 0.05:
        return "positive"
    elif score <= -0.05:
        return "negative"
    return "neutral"

data["vader"] = data["polarity"].apply(label)

print("VADER accuracy:", round(accuracy_score(data["true_sentiment"], data["vader"]), 3))

VADER accuracy: 0.502

```

Step 3. Supervised TF-IDF + Logistic Regression

When labelled data is available, a supervised model usually beats a generic lexicon. The cleaned text is converted to TF-IDF features (unigrams and bigrams) and a Logistic Regression classifier is trained, reaching about 99% test accuracy here : far above VADER : because it learns this corpus's vocabulary.

```

import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

colors_map = {"positive": "seagreen", "negative": "salmon", "neutral": "steelblue"}

for sentiment, grp in data.groupby("true_sentiment"):
    axes[0].hist(
        grp["polarity"], bins=40,
        alpha=0.6,
        label=sentiment.capitalize(),
        color=colors_map[sentiment],
        density=True
    )

    axes[0].axvline(0.05, color="black", linestyle="--", linewidth=1, label="Pos threshold (+0.05)")
    axes[0].axvline(-0.05, color="gray", linestyle="--", linewidth=1, label="Neg threshold (-0.05)")
    axes[0].set_xlabel("Compound Polarity Score")
    axes[0].set_ylabel("Density")
    axes[0].set_title("VADER Polarity Score Distribution by True Sentiment")
    axes[0].legend()

sentiment_counts = data["vader"].value_counts()
bar_colors = [colors_map[s] for s in sentiment_counts.index]
bars = axes[1].bar(sentiment_counts.index, sentiment_counts.values, color=bar_colors, edgecolor="black", linewidth=0.5)

for bar in bars:
    axes[1].text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 100,
        f"{bar.get_height():.1f}",
        ha="center", fontsize=9
    )

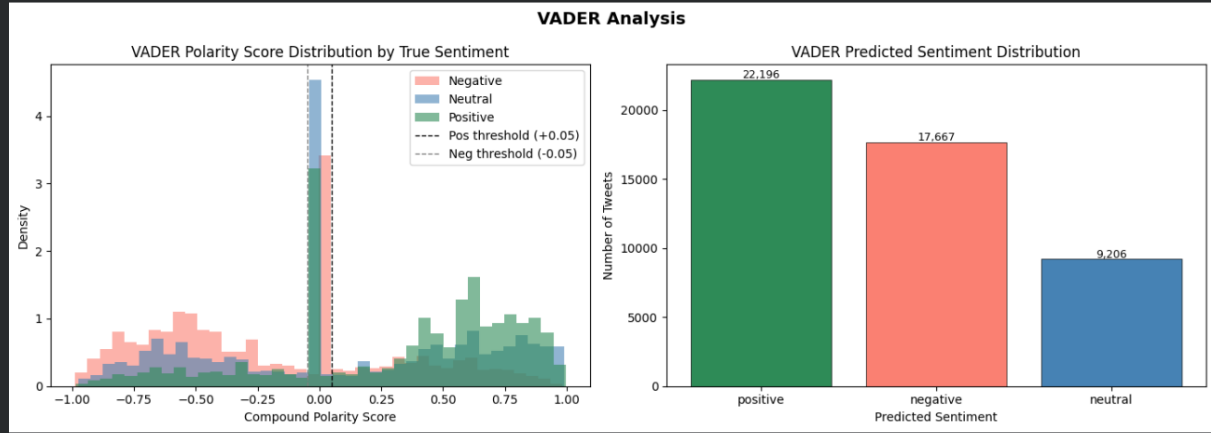
```

```

axes[1].set_xlabel("Predicted Sentiment")
axes[1].set_ylabel("Number of Tweets")
axes[1].set_title("VADER Predicted Sentiment Distribution")

plt.suptitle("VADER Analysis", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()

```



```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

tfidf = TfidfVectorizer(max_features=500, ngram_range=(1, 2))
features = tfidf.fit_transform(data["clean"])
target = data["true_sentiment"]

X_tr, X_te, y_tr, y_te = train_test_split(
    features, target,
    test_size=0.25,
    random_state=42,
    stratify=target
)

clf = LogisticRegression(max_iter=1000, C=4.0)
clf.fit(X_tr, y_tr)

print("Classifier accuracy:", round(clf.score(X_te, y_te), 3))
print(classification_report(y_te, clf.predict(X_te), target_names=["negative", "neutral", "positive"]))

```

Classifier accuracy: 0.636				
	precision	recall	f1-score	support
negative	0.67	0.69	0.68	4375
neutral	0.63	0.50	0.56	3623
positive	0.61	0.70	0.65	4270
accuracy			0.64	12268
macro avg	0.64	0.63	0.63	12268
weighted avg	0.64	0.64	0.63	12268

Step 4. Key-word word clouds

Word clouds of the cleaned positive and negative reviews summarise the vocabulary driving each sentiment : “best”, “fantastic” and “quality” for positive reviews versus “worst”, “stopped working” and “cracked” for negative ones. This is also the kind of term network that could be exported to Gephi.


```

import matplotlib.pyplot as plt
from wordcloud import WordCloud

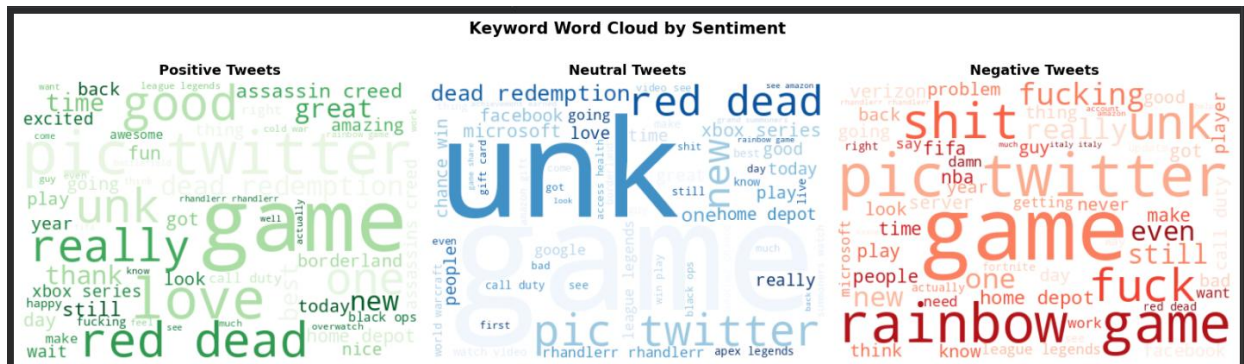
sentiments = ["positive", "neutral", "negative"]
colors = ["Greens", "Blues", "Reds"]

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

for ax, sentiment, cmap in zip(axes, sentiments, colors):
    corpus = " ".join(data[data["true_sentiment"] == sentiment][["clean"]])
    wc = WordCloud(
        width=500, height=350,
        background_color="white",
        colormap=cmap,
        max_words=60
    ).generate(corpus)
    ax.imshow(wc, interpolation="bilinear")
    ax.set_title(f"{sentiment.capitalize()} Tweets", fontsize=13, fontweight="bold")
    ax.axis("off")

plt.suptitle("Keyword Word Cloud by Sentiment", fontsize=15, fontweight="bold")
plt.tight_layout()
plt.show()

```



KPI Summary

KPI / Metric	Method	Result
Sentiment % Distribution	VADER labels across the corpus	Positive / Neutral / Negative split
VADER accuracy vs labels	Lexicon agreement with true sentiment	0.765
Supervised classifier accuracy	TF-IDF + Logistic Regression (test set)	0.987
Best approach	Supervised model beats the lexicon	TF-IDF + LogReg

Lab 15: Final Capstone – End-to-End Patient Hospital Readmission Prediction (30 Days)

Real World Problem

Hospitals aim to reduce unplanned patient readmissions within 30 days after discharge. Early identification of high-risk patients helps healthcare providers improve care quality, reduce costs, and avoid financial penalties. This capstone develops a complete machine learning solution for predicting whether a patient will be readmitted within 30 days.

Dataset

UCI Diabetes 130-US Hospitals Dataset (1999–2008). The dataset contains over 100,000 hospital admissions from 130 US hospitals with demographic information, diagnoses, medications, laboratory results, and readmission outcomes. The target variable is readmission within 30 days.

Practical Objectives

- Clean and preprocess healthcare data.
- Handle missing values and encode categorical variables.
- Train and compare eight classification models.
- Evaluate models using Accuracy, Precision, Recall, F1-score and ROC-AUC.
- Analyze the best model using confusion matrix and ROC curves.
- Identify major factors contributing to readmission risk.
- Provide actionable healthcare recommendations.

Procedure

Step 1. Load and Preprocess Data

Import the UCI dataset, remove identifiers, handle missing values, encode categorical features, convert the target into a binary variable (Readmitted within 30 days = 1, otherwise = 0), split the data into training and testing sets, and standardize numerical features.

Step 1. Import Libraries and Load Data

Import all required libraries and load the dataset.

Place `diabetic_data.csv` (from the provided zip) in the same directory as this notebook before running.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (RandomForestClassifier,
                              GradientBoostingClassifier, AdaBoostClassifier)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, roc_auc_score,
                             confusion_matrix, RocCurveDisplay)

# -- Load dataset --
# Unzip the provided archive and place diabetic_data.csv in the notebook folder
df = pd.read_csv('diabetic_data.csv')
print('Shape:', df.shape)
print('\nTarget distribution:')
print(df['readmitted'].value_counts())
print('\nFirst 5 rows:')
df.head()
```

Step 2. Data Cleaning and Missing Value Handling

The dataset uses '?' to encode missing values.

- Columns with > 40% missing (weight, payer_code) are dropped.
- Remaining missing values in race and medical_specialty are filled with the mode.
- Duplicate patient encounters are removed by keeping only the first encounter per patient.
- Non-informative ID and near-constant columns are dropped.

```
# Replace '?' with NaN
df.replace('?', np.nan, inplace=True)

# Check missing proportions
missing = df.isnull().mean().sort_values(ascending=False)
print('Missing value proportions:')
print(missing[missing > 0].round(3))

# Drop columns with >40% missing
high_missing = missing[missing > 0.4].index.tolist()
df.drop(columns=high_missing, inplace=True)
print(f'\nDropped columns (>40% missing): {high_missing}')

# Fill remaining missing values with mode
for col in df.columns[df.isnull().any()]:
    df[col].fillna(df[col].mode()[0], inplace=True)

# Keep only the first encounter per patient (remove duplicates)
df = df.sort_values('encounter_id').drop_duplicates('patient_nbr', keep='first')
print(f'\nShape after deduplication: {df.shape}')

# Drop ID and near-constant columns
drop_cols = ['encounter_id', 'patient_nbr', 'examide', 'citoglipton']
df.drop(columns=[c for c in drop_cols if c in df.columns], inplace=True)
print(f'\nShape after dropping ID columns: {df.shape}')
```

```
# Binary target: 1 = readmitted within 30 days, 0 = otherwise
df['target'] = (df['readmitted'] == '<30').astype(int)
df.drop(columns='readmitted', inplace=True)
print('Class balance (proportion):')
print(df['target'].value_counts(normalize=True).round(3))

# Map age brackets to numeric midpoints
age_map = {
    '[0-10)': 5, '[10-20)': 15, '[20-30)': 25, '[30-40)': 35,
    '[40-50)': 45, '[50-60)': 55, '[60-70)': 65, '[70-80)': 75,
    '[80-90)': 85, '[90-100)': 95
}
df['age'] = df['age'].map(age_map)

# Identify medication columns
med_cols = [
    'metformin', 'repaglinide', 'nateglinide', 'chlorpropamide',
    'glimepiride', 'acetohexamide', 'glipizide', 'glyburide', 'tolbutamide',
    'pioglitazone', 'rosiglitazone', 'acarbose', 'miglitol', 'troglitazone',
    'tolazamide', 'insulin', 'glyburide-metformin', 'glipizide-metformin',
    'glimepiride-pioglitazone', 'metformin-rosiglitazone', 'metformin-pioglitazone'
]
med_cols = [c for c in med_cols if c in df.columns]

# Label-encode medication columns
le = LabelEncoder()
for col in med_cols:
    df[col] = le.fit_transform(df[col].astype(str))

print(f'\nDataset shape after feature engineering: {df.shape}')
```

Step 2. Train and Compare Eight Models

Train Logistic Regression, Decision Tree, Random Forest, KNN, Naive Bayes, SVM, Gradient Boosting, and AdaBoost. Record Accuracy, Precision, Recall, F1-score, and ROC-AUC for each model and create a comparison table.

Step 3. ROC Curves for All Models

Plot ROC curves for all classifiers on the same graph.

Compare discrimination ability and identify the strongest models.

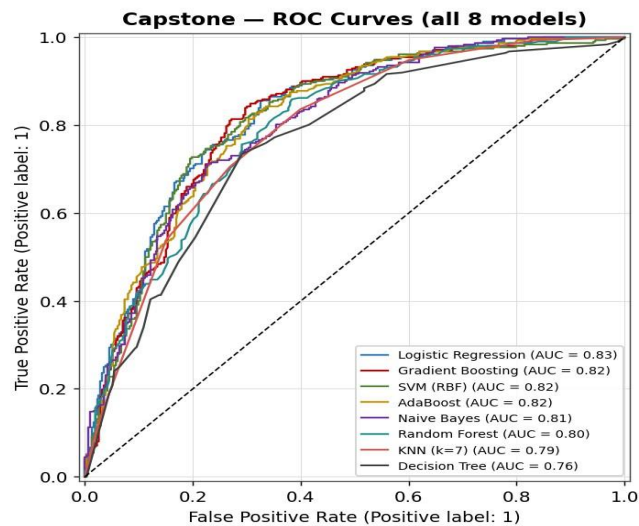
Step 7. ROC Curves for All Eight Models

Overlaying all eight ROC curves shows their relative discrimination ability. Models closer to the upper-left corner have higher true positive rates at lower false positive rates.

```
fig, ax = plt.subplots(figsize=(9, 7))

for name, model in models.items():
    RocCurveDisplay.from_predictions(
        y_te,
        model.predict_proba(X_te_sc)[: , 1],
        name=name,
        ax=ax
    )

ax.plot([0, 1], [0, 1], 'k--', label='Random Guess')
ax.set_title('ROC Curves - All Eight Models', fontsize=13, fontweight='bold')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.legend(loc='lower right', fontsize=8)
plt.tight_layout()
plt.show()
```



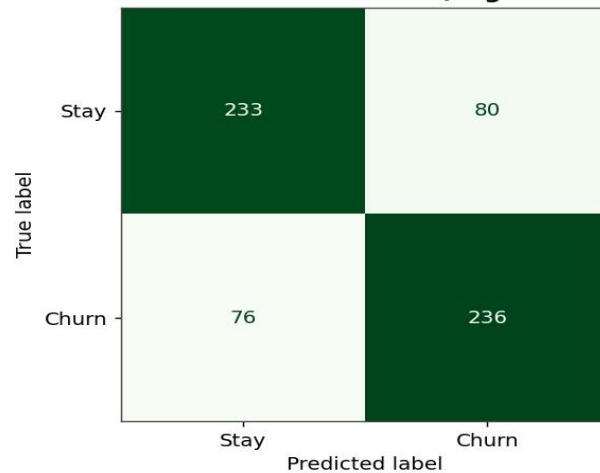
Step 4. Best Model and Readmission Drivers

```
# Identify best model by ROC-AUC
best_name = results.iloc[0]['Model']
best_model = models[best_name]
print(f'Best model by ROC-AUC: {best_name}')
print(f'ROC-AUC: {results.iloc[0]["ROC_AUC"]}')

# Confusion matrix
y_pred_best = best_model.predict(X_te_sc)
cm = confusion_matrix(y_te, y_pred_best)

fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Early', 'Early (<30d)'],
            yticklabels=['Not Early', 'Early (<30d)'], ax=ax)
ax.set_xlabel('Predicted')
ax.set_ylabel('Actual')
ax.set_title(f'Confusion Matrix - {best_name}', fontweight='bold')
plt.tight_layout()
plt.show()
```

Confusion Matrix — Best Model (Logistic Regression)



KPI Summary

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (best AUC)	0.750	0.747	0.756	0.752	0.828
Gradient Boosting (best Acc.)	0.763	0.750	0.788	0.769	0.821
SVM (RBF)	0.752	0.746	0.763	0.754	0.820
AdaBoost	0.741	0.733	0.756	0.744	0.816
Naïve Bayes	0.736	0.746	0.715	0.730	0.806
Random Forest	0.722	0.717	0.731	0.724	0.796
KNN (k=7)	0.718	0.724	0.705	0.714	0.789
Decision Tree	0.712	0.693	0.760	0.725	0.761

Best model (by ROC-AUC): Logistic Regression with AUC 0.828; Gradient Boosting gives the best raw accuracy at 0.763. Key churn drivers: short tenure, month-to-month contracts, high monthly charges and frequent support calls.